AN2401
Application Note

BLE 用户指导书

简介

本应用手册具体介绍了 BLE 协议栈相关流程、API 函数、参数设置、及 API 函数调用示例。

旨在减少研发工程师的开发时间，提高效率，缩短项目周期。

Freqchip Confidential

Version 1.2

修订版本

版本号	日期	更新内容
V1.0	2023.01.26	首版
V1.1	2025.01.03	增加 1.4 章节 Audio API 相关信息
V1.2	2025.03.19	增加 1.5 章节 L2CAP COC 相关信息

Freqchip Confidential

目录

1. BLE 协议栈 4
1.1 GAP 4
1.1.1 GAP 回调函数 4
1.1.2 广播 5
1.1.3 扫描 12
1.1.4 连接 13
1.2 GATT 15
1.2.1 GATT 角色 15
1.2.2 GATT Profile 层级 15
1.2.3 GATT 客户端基础流程 20
1.2.4 GATT 服务端基础流程 33
1.3 SMP 40
1.3.1 配对 40
1.3.2 加密 47
1.3.3 隐私管理 49
1.4 GAF 51
1.4.1 BAP--Unicast Audio 51
1.5 L2CAP COC 71
1.5.1 背景介绍 71
1.5.2 API 说明 73
联系信息 76

Freqchip Confidential

1. BLE 协议栈

FR5090 SDK 包含了完整的 BLE 协议栈，其中 controller 在 ROM 中实现，host 部分在 lib 中实现，提供了一组丰富的 API 供用户调用来完成蓝牙相关功能。

1.1 GAP

1.1.1 GAP 回调函数

设备连接成功、参数更新、搜到广播包等消息属于 GAP 消息，应用层需要用 gap_set_cb_func(gap_callback_func_t gap_evt_cb)函数来注册 GAP 接收函数来处理这些消息。
注册的 GAP 回调函数示例如下：

```
uint16_t gap_callback(struct gap_event *event)
{
switch(event->type){
    case GATT_EVT_PROFILED_ADDED:
        ...
        break;
    case GAP_EVT_ADV_START:
        ...
        break;
    case GAP_EVT_SCAN_END:
        ...
        break;
    case GAP_EVT_ADV_REPORT:  {
            ...
        }
        break;
    case GAP_EVT_MASTER_CONNECT:
            ...
        break;
    case GAP_EVT_SLAVE_CONNECT:
            ...
        break;
...
```

1.1.2 广播

1.1.2.1 Adv_handle

在创建广播之前，首先需要创建广播 handle。广播 handle 是后续设置广播参数和广播数据以及开启广播所必要的参数，每个 handle 可以保存一条广播链路的相关数据以及开启一个广播，创建多个 handle 可以同时开启多个广播：

```
adv_handle adv_1;
adv_1 = gap_adv_create();
```

1.1.2.2 传统广播

当设备开启传统广播流程时，应用层与 BLE 协议栈之间的交互流程如下所示：

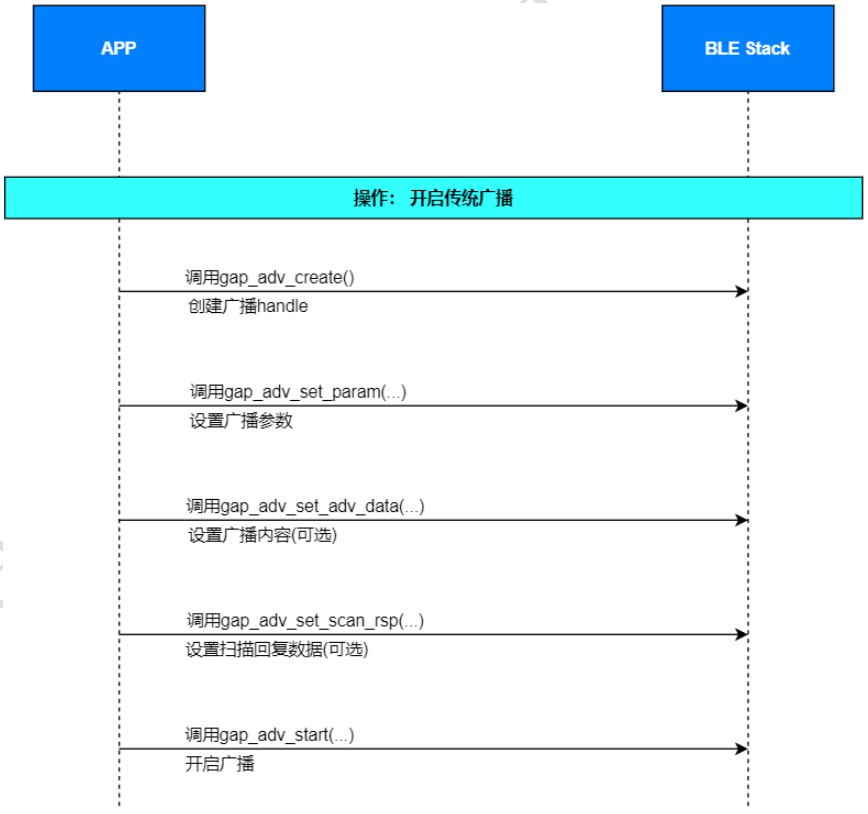


图 1-1 使能传统广播流程

1. 设置广播参数

下面以配置一个可被发现、可被连接、广播间隔为 40ms 的普通广播为例介绍如何设置广播参数：

```
gap_adv_param_t adv_param;
adv_param.adv_mode          = GAP_ADV_MODE_UNDIRECT;
adv_param.disc_mode         = GAP_ADV_DISC_MODE_GEN_DISC;
adv_param.own_addr_type     = GAP_ADDR_TYPE_STATIC;
adv_param.adv_chnl_map      = GAP_ADV_CHAN_ALL;
adv_param.filt_policy       = GAP_ADV_FILTER_SCAN_ANY_CON_ANY;
adv_param.phy_mode          = GAP_PHY_TYPE_LE_1M;
adv_param.adv_intv_min      = 0x40; //advertising min interval, in unit of 0.625ms
adv_param.adv_intv_max      = 0x40; //advertising max interval, in unit of 0.625ms
gap_adv_set_param(adv_1, &adv_param);
```

其中：

1. gap_adv_set_param(adv_1, &adv_param)中的 adv_1 为调用 gap_adv_create()创建的广播 handle。
2. adv_mode 参数用于设置广播类型，具体参见 gap_api.h 中枚举定义。当 adv_mode 为 DIRECT 类广播，如 GAP_ADV_MODE_DIRECT 时，disc_mode 参数需设置为 GAP_ADV_DISC_MODE_NON_DISC；当 adv_mode 设置为 GAP_ADV_MODE_BEACON 时，disc_mode 需设为 GAP_ADV_DISC_MODE_BEACON。
3. 当广播为传统广播(legacy advertising)时，phy mode 需为 GAP_PHY_TYPE_LE_1M。

对于已经开启的广播，用户需要先停止当前广播才能重新配置参数，否则将会配置失败。

2. 设置广播内容

当广播参数设置中 adv_mode 字段配置为 GAP_ADV_MODE_DIRECT 或 GAP_ADV_MODE_HDC_DIRECT 时，不需要设置广播数据，其他情况均需要对广播内容进行设置。传统广播最大数据段为 31 字节，其中 flag 字段由 host 协议栈内部根据广播参数生成，占据三个字节，因此留给用户配置的数据长度为 28 字节。广播的内容需要严格按照蓝牙协议规定的 length，type，data 格式进行填写，否则将会设置失败。

下面以设置一个广播名称为“frechip_5090”，自身服务 UUID 为 0xFFFE 的广播数据为例：

```
uint8_t adv_data[] =
{
    0x0D,                                           //length of name type
    GAP_ADVTYPE_LOCAL_NAME_COMPLETE,  //0x09, name type
    'f','r','e','c','h','i','p','_','5','0','9','0',       //name data
    0x03,                                           //length of uuid type
    GAP_ADVTYPE_16BIT_COMPLETE,       //0x03,service type,Service:More 16-bit UUIDS available
    0xFF,0xFE,                                      //uuid
};
gap_adv_set_adv_data(adv_1, adv_data, sizeof(adv_data));
```

Frechip Confiden

在需要更新广播内容时，用户无需暂停当前广播，可以直接调用该函数对内容进行更新。

3. 设置扫描回复内容

扫描回复（scan response）内容为设备在广播过程中收到扫描请求（scan request）时做出的回复。当广播参数设置中的 adv_mode 字段为 GAP_ADV_MODE_UNDIRECT 或 GAP_ADV_MODE_NON_CONN_SCAN 时，需要设置扫描响应数据，其他情况不需要设置。在传统广播中，扫描回复内容最长可以配置 31 字节。扫描回复的内容需要严格按照蓝牙协议规定的 length，type，data 格式进行填写，否则将会设置失败。

下面以设置一个扫描回复内容为设备自身 name 的 scan response 为例：

```
uint8_t scan_rsp_data[] =
{
    0x0D,                               //length of name type
    GAP_ADVTYPE_LOCAL_NAME_COMPLETE,  //0x09, name type
    'f','r','e','c','h','i','p','_','5','0','9','0',  //name data
};
gap_adv_set_scan_rsp(adv_1, scan_rsp_data, sizeof(scan_rsp_data));
```

在需要更新扫描回复内容时，用户无需暂停当前广播，可以直接调用该函数对内容进行更新。

4. 开启广播

调用 gap_adv_start 函数用于开启广播，输入参数是广播 handle、广播持续时间以及另一个用于扩展广播的参数，具体可查看 gap_api.h 文件。用于扩展广播的参数传统广播时不生效，默认设置为 0 即可；广播持续时间单位为 10ms，设置为 0 时表示广播不会自动停止，直到应用层主动停止广播或者设备与其他设备成功连接。

1.1.2.3 扩展广播

扩展广播的设置流程与传统广播的设置流程一致，二者的差异主要体现在广播参数的设置上。

1. 设置广播参数

下面以设置一个非定向、可连接、工作在 CODED PHY 上的 40ms 间隔的扩展广播为例，演示如何配置扩展广播的参数：

```
gap_adv_param_t adv_param;
adv_param.adv_mode          = GAP_ADV_MODE_EXTEND_CONN_UNDIRECT;
adv_param.disc_mode         = GAP_ADV_DISC_MODE_GEN_DISC;
adv_param.own_addr_type     = GAP_ADDR_TYPE_STATIC;
```

Freqchip Confidential

```
adv_param.adv_chnl_map      = GAP_ADV_CHAN_ALL;
adv_param.filt_policy       = GAP_ADV_FILTER_SCAN_ANY_CON_ANY;
adv_param.adv_intv_min      = 0x40;
adv_param.adv_intv_max      = 0x40;
adv_param.phy_mode          = GAP_PHY_TYPE_LE_CODED;
adv_param.max_skip          = 0x00;
adv_param.second_phy_mode   = GAP_PHY_TYPE_LE_1M;
adv_param.adv_sid           = 0x02;
gap_adv_set_param(adv_1, &adv_param);
```

其中：

1. adv_mode 参数用于设置广播类型，具体参见 gap_api.h 中枚举定义。当 adv_mode 为 DIRECT 类广播，如 GAP_ADV_MODE_EXTEND_CONN_DIRECT 时，disc_mode 参数需设置为 GAP_ADV_DISC_MODE_NON_DISC。
2. phy_mode 和 second_phy_mode 用于设定扩展广播 Primary Advertising 和 Secondary Advertising 采用的通信速率，phy_mode 需为 GAP_PHY_TYPE_LE_1M 或者 GAP_PHY_TYPE_LE_CODED，second_phy_mode 则多支持 2M PHY。
3. max_skip 参数用于控制扩展广播在发送 AUX_ADV_IND 数据包之前，可以跳过的 Advertising events 的最大数量。
4. adv_sid 参数表示本广播集合的应用层指定的 set_id 号，范围是 0~0x0F。

对于已经开启的广播，用户需要先停止当前广播才能重新配置参数，否则将会配置失败。

2. 设置广播内容

当 adv_mode 设置为 GAP_ADV_MODE_EXTEND_NON_CONN_SCAN 或 GAP_ADV_MODE_EXTEND_NON_CONN_SCAN_DIRECT 时，不需要设置广播数据。其余类型需设置：

```
uint8_t adv_data[] =
{
    0x0D,                           //length of name type
    GAP_ADVTYPE_LOCAL_NAME_COMPLETE,  //0x09, name type
    'f','r','e','c','h','i','p','_','5','0','9','0',  //name data
    0x03,                           //length of uuid type
    GAP_ADVTYPE_16BIT_COMPLETE,       //0x03,service type,Service:More 16-bit UUIDS available
    0xFF,0xFE,                      //uuid
};
gap_adv_set_adv_data(adv_1, adv_data, sizeof(adv_data));
```

3. 设置扫描回复内容

当 adv_mode 设置为 GAP_ADV_MODE_EXTEND_NON_CONN_SCAN 或 GAP_ADV_MODE_EXTEND_NON_CONN_SCAN_DIRECT 时，需要设置扫描回复数据，其余类型无需设置：

```
uint8_t scan_rsp_data[] =
{
    0x0D,                              //length of name type
    GAP_ADVTYPE_LOCAL_NAME_COMPLETE,   //0x09, name type
    'f','r','e','c','h','i','p','_','5','0','9','0',  //name data
};
gap_adv_set_scan_rsp(adv_1, scan_rsp_data, sizeof(scan_rsp_data));
```

4. 开启广播

扩展广播的开启方式与传统广播的开启方式一致，可参考传统广播。其中扩展广播开启时，需要设置 gap_adv_start 中的 max_adv_evt 参数，该参数用于设置设备在本次广播中可能发送的最大广播事件数目，为 0 则设备将一直广播，直到应用层主动停止广播或者设备与其他设备成功连接。

1.1.2.4 周期性广播

当设备开启周期性广播流程时，应用层与 BLE 协议栈之间的交互流程如下图所示：

Freqchip Confiden



图 1-2 开启周期性广播流程

1. 设置广播参数

```
gap_adv_param_t adv_param;
adv_param.adv_mode              = GAP_ADV_MODE_PER_ADV_UNDIRECT;
adv_param.disc_mode             = GAP_ADV_DISC_MODE_GEN_DISC;
adv_param.own_addr_type         = GAP_ADDR_TYPE_STATIC;
adv_param.adv_chnl_map          = GAP_ADV_CHAN_ALL;
adv_param.filt_policy           = GAP_ADV_FILTER_SCAN_ANY_CON_ANY;
adv_param.adv_intv_min          = 0x40;
adv_param.adv_intv_max          = 0x40;
adv_param.phy_mode              = GAP_PHY_TYPE_LE_1M;
adv_param.second_phy_mode       = GAP_PHY_TYPE_LE_1M;
adv_param.adv_sid               = 9;
adv_param.per_adv_intv_min      = 0x100;
adv_param.per_adv_intv_max      = 0x100;
gap_adv_set_param(adv_1, &adv_param);
```

其中：

1) per_adv_intv_min 和 per_adv_intv_max 表示周期性广播建立后，周期性广播内容的出现的最小最大时间间隔，单位：1.25ms。

2) 其他参数含义与章节 1.1.2.2 和章节 1.1.2.3 中描述一致。

2. 设置广播内容

广播内容设置方法与章节 1.1.2.2 中描述一致。

3. 设置周期性广播数据

周期性广播的数据采用下面的方式进行设置：

```
uint8_t period_data[] =
{
    0x0D,                               //length of name type
    GAP_ADVTYPE_LOCAL_NAME_COMPLETE,  //0x09, name type
    'f','r','e','c','h','i','p','_','5','0','9','0',  //name data
};
gap_adv_set_period_data(adv_1, period_data, sizeof(period_data));
```

4. 开启广播

扩展广播的开启方式与传统广播的开启方式一致，可参考章节 1.1.2.2。

1.1.2.5 相关 GAP 事件

与广播相关的 GAP 事件主要有以下几种：

1. GAP_EVT_ADV_SET_PARAM：设置广播参数完成，需根据上报的 status 参数判断是否设置成功。
2. GAP_EVT_ADV_SET_ADV_DATA：设置广播数据完成。
3. GAP_EVT_ADV_SET_SCAN_RSP：设置扫描回复数据完成。
4. GAP_EVT_ADV_SET_PER_DATA：设置周期性广播数据完成。
5. GAP_EVT_ADV_START：广播开始，在调用 gap_adv_start 后产生。
6. GAP_EVT_ADV_END：广播结束，该事件在主动调用 gap_adv_stop、广播超时、设备被连接三种情况下会产生。
7. GAP_EVT_SLAVE_CONNECT：正在广播可连接数据的设备(作为从机)在被连接上之后，应用层 GAP 回调函数会接收到该事件。

1.1.2.6 相关 API

关闭广播：gap_adv_stop

获取当前广播状态：gap_get_adv_state

Freqchip Confidential

1.1.3 扫描

1.1.3.1 传统扫描

当设备开启扫描流程时，应用层与 BLE 协议栈之间的交互流程如下：

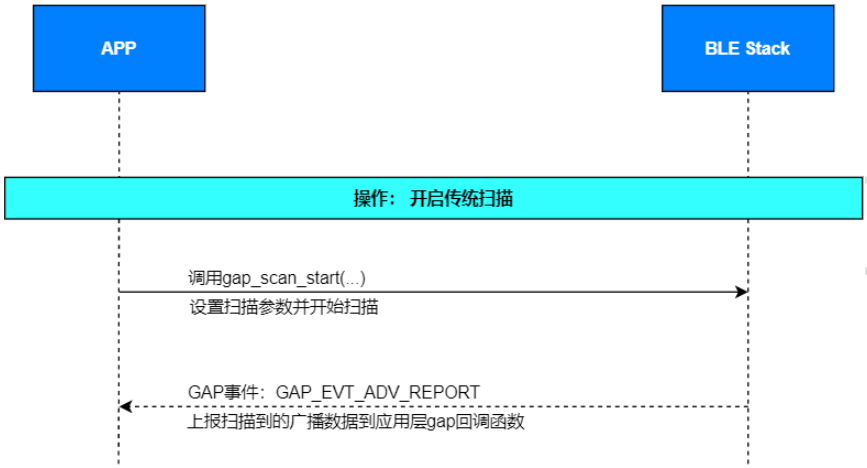


图 1-3 传统扫描流程

以下示例代码演示如何开启普通主动扫描：

```
gap_scan_param_t scan_param;
scan_param.scan_type          = GAP_SCAN_TYPE_GEN_DISC;
scan_param.own_addr_type      = GAP_ADDR_TYPE_STATIC;
scan_param.dup_filt_pol       = 0;
scan_param.scan_intv          = 32;  //scan event on-going time
scan_param.scan_window        = 20;
scan_param.duration           = 1500; //15s (in unit of 10ms)
scan_param.phy_mode           = GAP_PHY_TYPE_LE_1M;
gap_scan_start(&scan_param);
```

其中：

1. scan_type 指示扫描的模式：GAP_SCAN_TYPE_GEN_DISC 与 GAP_SCAN_TYPE_LIM_DISC 表示主动扫描，这种模式会根据 ADV 类型选择发送 scan request 数据包用来获取 scan response；GAP_SCAN_TYPE_OBSERVER 与 GAP_SCAN_TYPE_SEL_OBSERVER 表示被动扫描，这种模式只会处于被动监听状态，其中 SEL_OBSERVER 类型只会接收来自白名单的广播数据。
2. dup_filt_pol 表示对扫描到的内容上报前是否进行重复性过滤检查。
3. scan_window 和 scan_intv 分别表示扫描窗口和扫描间隔，单位为 625us。

4. duration 表示整个扫描动作持续时间，时间到后自动停止，单位 10ms。如果设置为 0，表示扫描动作不会自动停止。

1.1.3.2 相关 GAP 事件

与扫描相关的 GAP 事件主要有以下几种：

1. GAP_EVT_ADV_REPORT，该事件用于上报扫描到的广播数据
2. GAP_EVT_SCAN_START，该事件表示设备开始扫描，调用 gap_scan_start 后产生。
3. GAP_EVT_SCAN_END，该事件在调用停止扫描函数 gap_scan_stop，或者扫描时间到时会产生。

1.1.4 连接

1.1.4.1 主机发起连接

当设备发起连接流程时，应用层与 BLE 协议栈之间的交互流程下图所示：



图 1-4 连接建立流程

以下代码展示如何发起一个传统连接：

```
gap_conn_param_t conn_req;
memcpy(&conn_param.peer_addr.addr, &event->param.adv_rpt->addr.addr, 6);
conn_req.peer_addr.addr_type    = 0;
conn_req.own_addr_type          = GAP_ADDR_TYPE_STATIC;
conn_req.phy_mode               = GAP_PHY_TYPE_LE_1M;
```

```
conn_req.scan_intv          = 32;
conn_req.scan_window        = 32;
conn_req.conn_intv_min      = 30;
conn_req.conn_intv_max      = 30;
conn_req.slave_latency      = 0;
conn_req.supervision_to     = 300;
conn_req.ce_len_min         = 2;
conn_req.ce_len_max         = 2;
gap_conn_start(&conn_req);
```

其中：

1. peer_addr.addr 表示要连接的 BLE 设备的 mac 地址。
2. peer_addr.addr_type 表示要连接的 BLE 设备的 mac 地址类型。0 表示 public addr；1 表示 private addr。
3. conn_intv_max 和 conn_intv_min 表示连接建立后采用的握手基础间隔时间，单位：1.25ms，两者必须大于 7.5ms。
4. conn_to 表示本次发送连接请求的持续时长，0 表示设备将一直向对端设备发送连接请求，直到连接成功或者用户调用 gap_conn_stop()主动停止连接；
5. slave_latency 表示连接建立后 slave 设备采用的 latency 参数。
6. supervision_to 表示连接建立后 slave 设备采用的超时断开时间。单位：10ms。
7. ce_len_max 和 ce_len_min 表示主机的调度间隔，用于连接多个设备时，控制主机与多个从机的通信调度间隔，调控吞吐量。 单位：0.625ms。

1.1.4.2 其他

1. 断开连接

采用如下函数可以将已经建立的连接断开。该函数对于工作在主机或者从机模式下均适用。

```
ble_api_err_t gap_disconnect(uint8_t conidx);
```

2. 取消当前连接

在连接建立过程中，用户可以调用下面的函数来取消本次连接。

```
ble_api_err_t gap_conn_stop(void);
```

1.1.4.3 相关 GAP 事件

与连接相关的 GAP 事件主要由以下几个：

Freqchip Confidential

1. GAP_EVT_MASTER_CONNECT，作为主机，成功连接上从机。在发起主动连接请求，连上对端 BLE 设备后，协议栈会产生主动连接成功建立事件。
2. GAP_EVT_SLAVE_CONNECT，作为从机，成功与主机连接。在发送可连接广播后，收到主机发送的连接请求且成功连上主机时，会产生该事件。
3. GAP_EVT_CONN_END，在以下两种情况时：调用停止主动连接函数或者连接建立后，会产生主动连接动作停止事件。
4. GAP_EVT_DISCONNECT，设备断开连接，不论主机与从机，在与其他设备断开连接后，都会产生该事件。

1.2 GATT

通用属性规范 GATT（Generic Attribute Profile）用于两个连接设备之间的数据通信。在 BLE GATT 层中，数据以特征（Characteristic）的形式进行传输和存储。

1.2.1 GATT 角色

从 GATT 的角度来看，当两个设备处于连接状态时，一个设备作为 GATT 服务端，另一个设备作为 GATT 客户端。这种角色的分配与 GAP 层设备角色（外围设备、中央设备）无关。一个外围设备既可以充当 GATT 客户端，也可以充当 GATT 服务端；一个中央设备同样既可以充当 GATT 客户端，也可以充当 GATT 服务端。

- GATT 客户端：设备发起命令、请求并接收响应、通知和指示。
- GATT 服务端：设备接收命令、请求并发出响应、通知和指示。

1.2.2 GATT Profile 层级

Profile 是一个抽象的概念，可以将其理解为“应用场景、功能、通信格式”固定好的一种规范，一个 profile 由一个或多个相关的服务构成；一个服务可包含一个或者多个特征；一个特征至少包含两个属性，包括特征声明和特征值。下面章节将具体描述属性、特征和服务。

Freqchip Confidential

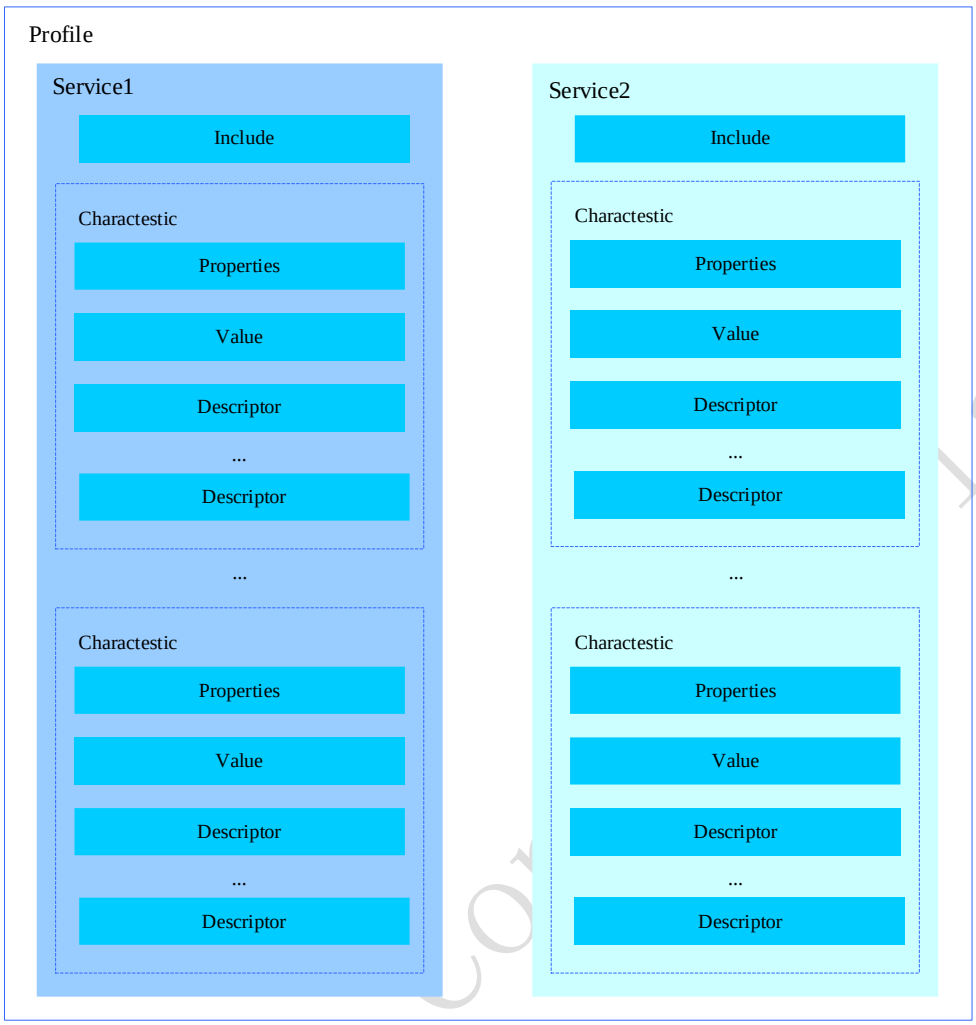


图 1-5 服务、特征、属性层级关系

1.2.2.1 属性（Attribute）

在低功耗蓝牙中，特征可以看作是一组属性信息的集合，包括特征声明、特征值以及特征描述符，数据以属性的形式进行交互。属性主要包括以下几个部分：

- 属性句柄：句柄是属性在 GATT 属性表中的索引，每个属性都有唯一的句柄。
- 属性类型：表示数据代表的事物，通常是 Bluetooth SIG 规定或由用户自定义的 UUID（通用唯一标识符）。
- 属性值：属性的数据值。
- 属性许可权限：规定了 GATT 客户端设备对属性的访问权限，包括是否能访问和怎样访问。

1.2.2.2 特征（Characteristic）

一个典型的特征由以下属性组成：

- 特征声明：描述特征的性质、存储位置（句柄）、类型。

Freqchip Confidential

- 特征值：特征的数据值。
- 特征描述：描述特征的附加信息或配置。

1.2.2.3 服务（Service）

GATT 服务是一系列特征的集合，例如，心率服务包括心率测量特征和身体位置特征等。常见的服务有：

- 通用访问服务（Generic Access Service）：该服务包括设备及访问信息等，比如设备名称、供应商标志、产品标志等。该服务定义的特征有：设备名称（Device Name）、外观（Appearance）、外设优先连接参数（Peripheral Preferred Connection Parameters）等。
- 通用属性服务（Generic Attribute Service）：该服务主要用于服务端通知所连接的客户端其提供的服务发生了变化。该服务包含 Service changed 特征。

上述两个服务在 BLE 协议栈初始化时，默认添加到服务端的数据库中。但用户可以在协议栈初始化后添加用户服务之前调用 gatt_delete_gap_gatt_svc 函数删除这两个默认的服务。

1.2.2.4 属性表定义及建立属性表

低功耗蓝牙的通信就是通过对属性表中的属性进行读写等操作来实现的，本节介绍如何定义这些属性来构成一个完整的服务(属性表)。

每个 Service 必须定义一个属性表并通过服务添加函数 gatt_add_service 将其传递给协议栈。属性表建立的示例代码如下：

```
#define SP_SVC_UUID             0xFFF0
#define SP_CHAR1_UUID           0xFFF3
#define SP_CHAR2_UUID           {0xba, 0x5c, 0xFF, 0xF4, 0x04, 0xa3, 0x40, 0x71, 0xa0, 0xb5,
0x35, 0x85, 0x3e, 0xb0, 0x83, 0x07}
// Element index of gatt service table
enum
{
    SP_IDX_SERVICE,
    SP_IDX_CHAR1_DECLARATION,
    SP_IDX_CHAR1_VALUE,
    SP_IDX_CHAR1_USER_DESCRIPTION,
    SP_IDX_NB,
};

typedef struct gatt_uuid {
```

Confidential

```
    uint8_t size;           //!< Length of UUID (2 or 16 bytes UUIDs: UUID_SIZE_2 or
UUID_SIZE_16).
    uint8_t p_uuid[16];     //!< Pointer to uuid, could be 2 or 16 bytes array.
}gatt_uuid_t;

typedef struct gatt_attribute {
    struct gatt_uuid uuid;  //!< Attribute UUID
    uint16_t prop;          //!< Attribute properties, see @GATT_PROP_BITMAPS_DEFINES
    uint16_t max_size;      //!< Attribute data maximum size
    uint8_t *p_data;        //!< Attribute data pointer
}gatt_attribute_t;
static const uint8_t sp_svc_uuid[]   = UUID16_ARR(SP_SVC_UUID);
static const gatt_attribute_t batt_att_table[] =
{
    [SP_IDX_SERVICE] = { { UUID_SIZE_2, UUID16_ARR(GATT_PRIMARY_SERVICE_UUID)},
        GATT_PROP_READ,  UUID_SIZE_2, (uint8_t *) sp_svc_uuid,
    },
    [SP_IDX_CHAR1_DECLARATION] = { { UUID_SIZE_2, UUID16_ARR(GATT_CHARACTER_UUID)},
        GATT_PROP_READ,0, NULL,
    },
    [SP_IDX_CHAR1_VALUE] = { { UUID_SIZE_2, UUID16_ARR(SP_CHAR1_UUID)},
        GATT_PROP_READ | GATT_PROP_NOTI, SP_CHAR1_VALUE_LEN,NULL,
    },
    [SP_IDX_CHAR1_USER_DESCRIPTION] = { {UUID_SIZE_2, UUID16_ARR(GATT_CHAR_USER_DESC_UUID)},
        GATT_PROP_READ | GATT_PROP_WRITE, SP_CHAR1_USER_DESC_LEN,NULL,
    },
};
```

说明：

- 属性表 simple_profile_att_table [] 是一个数组。数组变量类型为结构体类型 gatt_attribute_t。数组里每一个变量构成属性表里的一个属性。
- 结构体类型 gatt_attribute_t 里各元素解释如下：
  a) uuid 表示这条属性的 UUID。uuid.size 是 UUID 的长度，值可以是 16bits 或者是 128bits，用宏 UUID_SIZE_2 和 UUID_SIZE_16 表示。uuid.p_uuid 是 UUID 的内容。
  b) prop 表示这条属性的读写权限。profile 属性权限宏定义参见 gatt_api.h 宏定义组

GATT_PROP_BITMAPS_DEFINES 里的定义，如下：

```
#define GATT_PROP_BROADCAST          (1<<0)   //!< Attribute is able to broadcast
#define GATT_PROP_READ               (1<<1)   //!< Attribute is Readable
#define GATT_PROP_WRITE_CMD          (1<<2)   //!< Attribute supports write with no response
#define GATT_PROP_WRITE_REQ          (1<<3)   //!< Attribute supports write request
#define GATT_PROP_NOTI               (1<<4)   //!< Attribute is able to send notification
#define GATT_PROP_INDI               (1<<5)   //!< Attribute is able to send indication
#define GATT_PROP_SIG_WRTIE          (1<<6)   //!< Attribute supports authenticated signed write
#define GATT_PROP_EXTEND_PROP        (1<<7)   //!< Attribute supports extended properities
#define GATT_PROP_READ_UNAUTH        (1<<8)   //!< Attribute read requires Unauthentication
#define GATT_PROP_READ_AUTHEN        (2<<8)   //!< Attribute read requires Authentication
#define GATT_PROP_READ_SECURE_CONN   (3<<8)   //!< Attribute read requires Secure Connection
#define GATT_PROP_WRITE_UNAUTH       (1<<10) //!< Attribute write requires Unauthentication
#define GATT_PROP_WRITE_AUTHEN       (2<<10) //!< Attribute write requires Authentication
#define GATT_PROP_WRITE_SECURE_CONN  (3<<10) //!< Attribute write requires Secure Connection
#define GATT_PROP_NI_UNAUTH          (1<<12) //!< Attribute notification or indication requires
                                             Unauthentication
#define GATT_PROP_NI_AUTHEN          (2<<12) //!< Attribute notification or indication requires
                                             Authentication
#define GATT_PROP_NI_SECURE_CONN       (3<<12) //!< Attribute notification or indication
                                               requires Secure Connection
```

常用的读写权限是第 2~6 行，GATT_PROP_READ_UNAUTH 及后续的属于安全管理方面。PROP 可以通过或的方式设置属性支持多条权限。如：GATT_PROP_READ | GATT_PROP_NOTI 表示属性同时支持 notify 和 read； GATT_PROP_READ | GATT_PROP_READ_UNAUTH 表示属性可读，但需要进行配对加密后，才能正常进行读流程。

a) max_size 表示该属性支持的最大数据长度。

b) p_data 表示该属性的属性值，为指针类型。

- 给 UUID 为 GATT_PRIMARY_SERVICE_UUID 的属性变量赋值时，max_size 和 p_data 分别表示服务组 UUID 的长度和 UUID 的内容。示例属性表中，该属性元素 max_size 为 UUID_SIZE_2，p_data 为 (uint8_t *)sp_svc_uuid。分别表示这个服务组的 UUID 长度是 16bits，UUID 的值是 sp_svc_uuid 的值。

1.2.3 GATT 客户端基础流程

1.2.3.1 客户端

GATT 客户端主要向服务端发起读写命令和请求并接收服务端的响应、指示和通知。

GATT 客户端没有属性表，它主要用于获取属性信息而不是提供属性信息和服务。对属性表属性的读写操作通过使用 gatt_api.h 定义的的 API 接口完成。大部分 GATT 客户端的 API 被调用之后，能通过客户端事件回调函数返回结果，包括读取的属性值、写是否完成状态、指示等。关于 API 的详细描述，请参考 gatt_api.h 内函数定义说明。

在 FR5090 的协议栈实现中，我们提供了两种客户端与服务端的交互方式，分别为标准流程和简化流程。两种方式可同时使用，也可单独使用，其中简化流程用于客户端仅需与服务端进行特征值内容读写的情况。两种流程均需要调用 gatt_add_client 函数注册客户端的 profile，在注册 profile 的时候简化流程会额外需要添加一个关心的服务端 uuid 数组，该 uuid 数组只能含有特征值的 uuid。后续客户端对服务端进行读写等操作时，标准流程以 handle 为参数，简化流程则以 uuid 数组索引为参数。详细内容请参见 1.2.3.3 节。

1.2.3.2 注册客户端

作为客户端的设备需要向协议栈注册客户端的 profile，该 profile 是后续客户端与服务端正常交互的前提。注册实例如下。

```
#define SP_CHAR1_UUID           0xFFF1
#define SP_CHAR2_UUID           0xFFF2
#define SP_CHAR3_UUID           0xFFF4
const gatt_uuid_t client_att_tb[] =
{
    [0]  ={ UUID_SIZE_2, UUID16_ARR(SP_CHAR1_UUID)},
    [1]  ={ UUID_SIZE_2, UUID16_ARR(SP_CHAR2_UUID)},
    [2]  ={ UUID_SIZE_2, UUID16_ARR(SP_CHAR3_UUID)},
};
uint8_t client_id;
static uint16_t gatt_callback(struct gatt_msg *p_msg)
{
    switch(p_msg->msg_evt)
    {
        case GATTC_MSG_CMP_EVT:
            switch(p_msg->param.gatt_op_cmp.operation) {
            case GATT_OP_PEER_SVC_DISC_END:
                LOG_INFO(NULL, "dicovery peer service end, status:%02X\r\n",
```

```
                        p_msg->param.gatt_op_cmp.status);
                break;
            case GATT_OP_PEER_SVC_REGISTERED:
                LOG_INFO(NULL, "peer char_hdl registered\r\n");
                break;
            default:
                break;
        }
        break;
    }
}
void app_ble_add_client(void)
{
    struct gatt_client client;
    client.p_att_tb = client_att_tb; //Can be set to NULL, when set to NULL, client.att_nb is 0
    client.att_nb = 2;
    client.gatt_msg_handler = gatt_callback; //set GATT event callback
    client_id = gatt_add_client(&client);
}
```

说明：

- client_att_tb[]是一个 uuid 的数组，适用于简化流程，记录用户真正需要使用的特征值属性的 uuid。调用扫描服务函数后，协议栈能保存这些 uuid 属性对应的 handle 号。
- client_id 是定义的记录这个客户端 profile 的 id 号，后续对服务端进行读写操作时需要用到这个数值。
- gatt_callback，定义的是客户端 profile 对应的事件回调函数，协议栈需要上报 GATT 事件时，会调用该函数。
- app_ble_add_client 里面定义了客户端 profile 的变量，将回调函数赋值给该变量后，调用 gatt_add_client 函数将该客户端 profile 添加到协议栈内部去。其中若客户端存在使用简化流程的情况，则需要将 uuid 数组(上述中的 client_att_tb)和 uuid 数量赋值给变量的参数 p_att_tb 以及 att_nb；对于标准流程，这两个参数没有影响。
- 客户端 profile 添加成功后，协议栈上传 GAP 事件 GATT_EVT_PROFILE_ADDED 到 GAP 回调函数里。
- 可在程序运行的任何阶段添加客户端 profile，但最好在程序初始化阶段时添加。

1.2.3.3 客户端与服务端交互流程

1.标准流程

标准流程中与服务端的交互均通过 handle 进行。在连接建立好之后，用户可在 GAP 回调函数中 GAP_EVT_MASTER_CONNECT 事件下调用 gatt_discovery_all_peer_svc 函数扫描服务端服务，之后会产生 GATT 回调事件 GATTC_MSG_SVC_REPORT，该事件会产生 N 次，N 取决于服务端的服务数量，每次事件会上报一个服务的 start_handle、end_handle 以及该服务的 uuid。用户利用上报的 start_handle、end_handle 以及特征发现函数 gatt_discovery_peer_char 可以扫描每个服务里包含的特征，调用 gatt_discovery_peer_char 函数将产生 GATT 回调事件 GATTC_MSG_CHAR_REPORT，该事件会产生 N 次，N 取决于扫描 handle 范围内特征的数量，每次事件会上报一个特征的 handle、特征值的 handle 等，具体可参考 gatt_api.h 中的结构体 gatt_char_report_t。了解服务端特征值 handle 后，客户端利用该 handle 可进行后续的读写等操作。具体流程可参考下图。

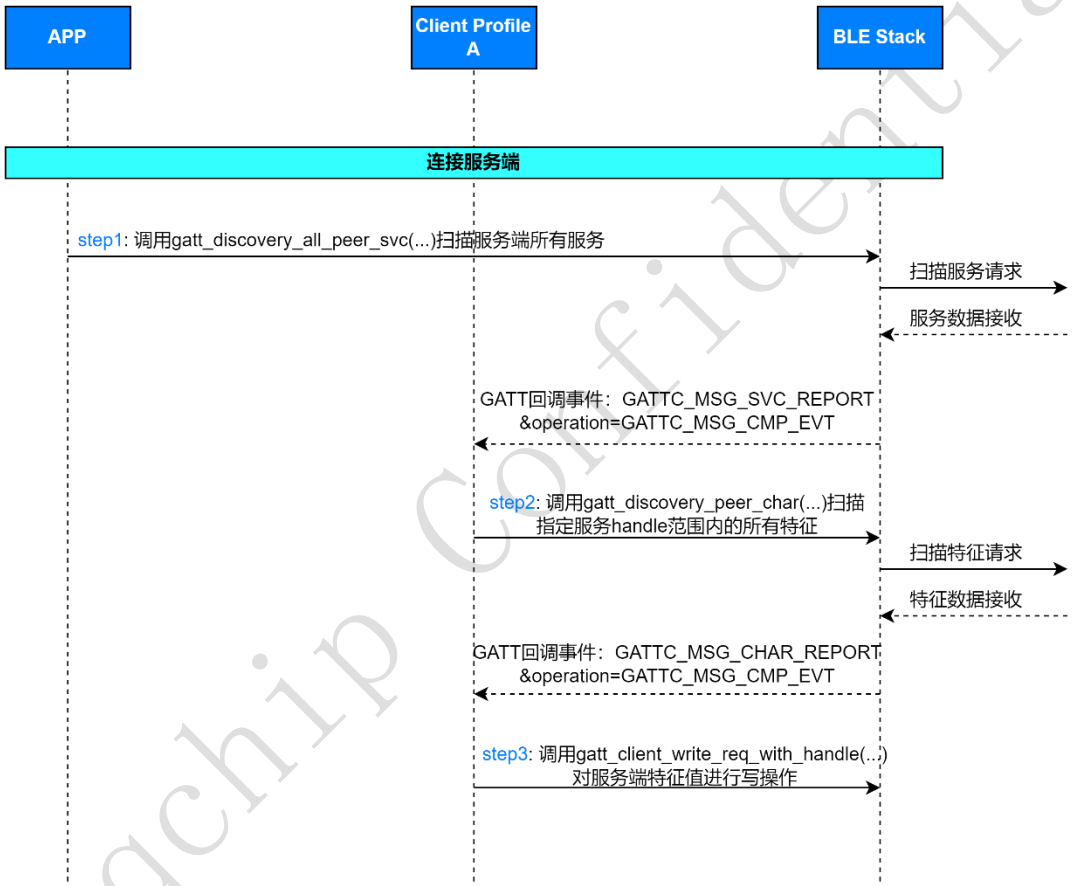


图 1-6 客户端标准流程

2.简化流程

简化流程中与服务端的交互通过 uuid 数组进行。该流程只针对于客户端仅需操作服务端特征值属性的情景，若想对其他属性(如特征描述)进行操作，请参考标准流程。为了简化应用层的服务发现流程，用户需要向协议栈注册一个应用层关心的服务端 uuid 数组，这个 uuid 数组包含了客户端可能会发起读、写、接收通知等功能会涉及到的属性的 uuid。在连接建立好之后，用户可在 GAP 回调函数中 GAP_EVT_MSATER_CONNECT 事件下调用 gatt_discovery_all_peer_svc 函数，控制协议栈去发现服务端数据库是否包含有初始化时注册的 uuid，若包含，则协议栈将会保存 uuid 对应特征的 handle，后续用户即可通过创建的 uuid 数组中属性的索引(即数组的下标)进行交互。具体流程可参考下图。

freqchip Confidential

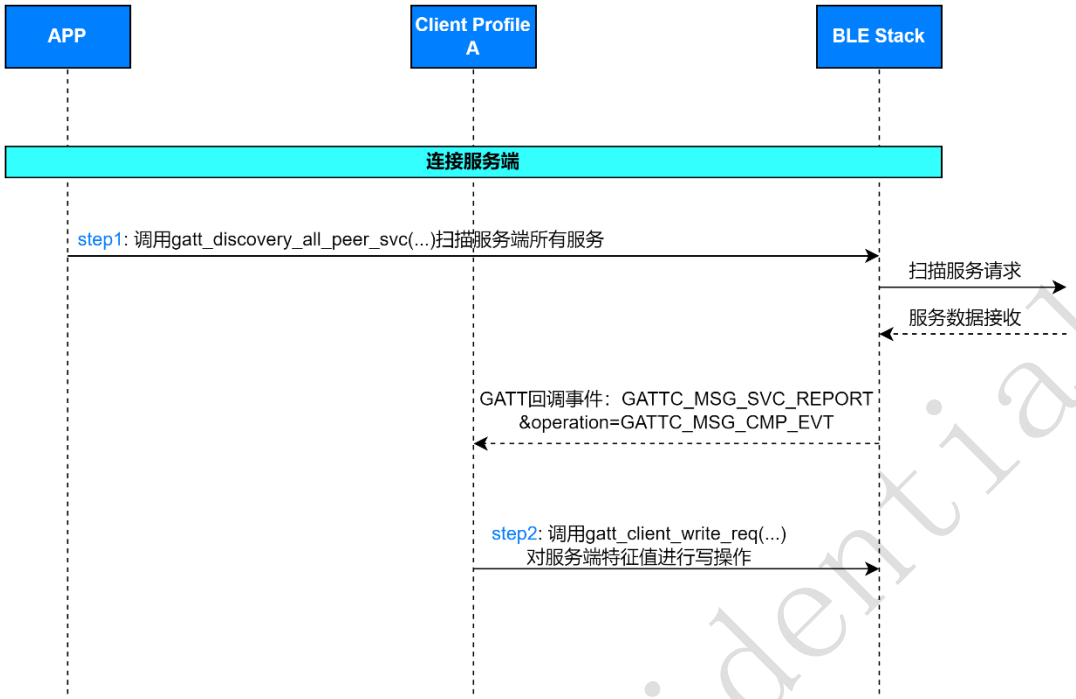


图 1-7 客户端简化流程

1.2.3.4 客户端 profile 回调函数

为了处理 profile 属性的读写操作，profile 需要定义自己的读写回调函数。回调函数接收协议栈上传的关于这个 profile 的所有消息。以下面 profile 的回调函数实例来解释。

```
static uint16_t simple_central_msg_handler(struct gatt_msg *p_msg)
{
    switch(p_msg->msg_evt)
    {
        case GATTC_MSG_NTF_REQ:
            show_reg(p_msg->param.msg.p_msg_data,p_msg->param.msg.msg_len,1);
        break;

        case GATTC_MSG_READ_IND:
            show_reg(p_msg->param.msg.p_msg_data,p_msg->param.msg.msg_len,1);
        break;

        case GATTC_MSG_SVC_REPORT:
        {
            gatt_svc_report_t *svc_dis =
        (gatt_svc_report_t *)(p_msg->param.gatt_data.p_msg_data);
      gatt_uuid_t svc_uuid; memcpy(&svc_uuid, &(svc_dis->uuid), sizeof(gatt_uuid_t));
     LOG_INFO(NULL,"s_hdl:0x%02X,e_hdl:0x%02X\r\n",svc_dis->start_hdl,svc_dis->end_hdl);
```

Confidential

```
        }
        break;

        case GATTC_MSG_CHAR_REPORT:
        {
            gatt_char_report_t *char_dis =
            (gatt_char_report_t *)(p_msg->param.gatt_data.p_msg_data);
            gatt_uuid_t char_uuid;
            memcpy(&char_uuid,&(char_dis->uuid), sizeof(gatt_uuid_t));
            LOG_INFO(NULL, "char_value_hdl:0x%02X\r\n", char_dis->val_hdl);
        }
        break;

        case GATTC_MSG_CMP_EVT:
        {
            if(p_msg->param.gatt_op_cmp.operation == GATT_OP_PEER_SVC_REGISTERED)
            {
                uint16_t att_handles[3];
                memcpy(att_handles,p_msg->param.gatt_op_cmp.arg,6);

                gatt_client_read_param_t read_att;
                read_att.conidx     = p_msg->conn_idx;
                read_att.att_idx    = 1;
                read_att.client_id = client_id;
                gatt_client_read(&read_att);
            }
            else if(p_msg->param.gatt_op_cmp.operation == GATT_OP_PEER_SVC_DISC_END)
            {
                LOG_INFO(NULL, "dicovery peer service end, status:0x%02X\r\n",
                            p_msg->param.gatt_op_cmp.status);
            }
        }
        break;
        default:
           break;
    }
    return 0;
}
```

说明：

- Profile 的 GATT 事件回调函数会传入一个消息变量。该变量包含了回调事件的类型、连接号、消息对应的 handle 号、消息的内容。

- GATT 的事件回调函数根据上传的消息的类型分别处理，函数以 return 0 结尾。
- GATTC_MSG_NTF_REQ 事件表示客户端接收到服务端设备发过来的 notification 消息，示例中做了打印处理。
- GATTC_MSG_READ_IND 事件表示客户端对服务端设备进行 read 操作后，接收到服务端设备发过来的 read response 数据，示例中做了打印处理。
- GATTC_MSG_SVC_REPORT 事件表示客户端对服务端进行扫描服务操作后，接收到的服务端设备发过来的服务扫描回复内容，包含一个服务的 uuid，起始 handle，结束 handle。
- GATTC_MSG_CHAR_REPORT 事件表示客户端对服务端进行扫描特征操作后，接收到的服务端发过来的特征扫描回复内容，包含一个特征的 uuid，特征 handle，特征值的 handle 以及该特征值的许可属性。
- GATTC_MSG_CMP_EVT 事件表示针对客户端 profile 的某个操作完成。
  p_msg->param.gatt_op_cmp.operation 代表操作的类型，类型一共有以下几种，在 gatt_api.h 内宏定义 GATT_OPERATION_NAME_DEFINES。

```
#define GATT_OP_NOTIFY                  0x01    //!< GATT notification operation
#define GATT_OP_INDICA                  0x02    //!< GATT indication operation
#define GATT_OP_PEER_SVC_REGISTERED     0x03    //!< Used with GATTC_CMP_EVT, GATT peer device
                                                //!< service registered
#define GATT_OP_WRITE_REQ               0x05    //!< GATT wirte request operation
#define GATT_OP_WRITE_CMD               0x06    //!< GATT wirte command operation, write
                                                //!< without response
#define GATT_OP_READ                    0x07    //!< GATT read operation
#define GATT_OP_PEER_SVC_DISC_END       0x08    //!< Used with GATTC_CMP_EVT, GATT peer device
                                                //!< service discovery is ended
#define GATT_OP_PEER_CHAR_DISC_END      0x09    //!< Used with GATTC_CMP_EVT, GATT peer device
                                                //!< characteristic discovery is ended
#define GATT_OP_PEER_DESC_DISC_END      0x0A    //!< Used with GATTC_CMP_EVT, GATT peer device
                                //!< characteristic describtion discovery is //!< ended
```

- 对客户端 profile 来说，除 0x01 和 0x02 外，其余操作类型均为客户端所有
  a) 0x03 代表，扫描服务步骤中扫描对端服务动作完成，并将关注的特征值(注册客户端时添加的 uuid 数组)的 handle 号注册到协议栈内部。此时，gatt_op_cmp.arg 指向的是客户端 uuid 数组中的 uuid 对应在服务端的 handle 号；gatt_op_cmp.arg_len 为 uuid 数组数组元素数量*2 的字节长度。
  b) 0x05 表示，对服务端属性进行写操作（需要 response）完成。
  c) 0x06 表示，对服务端属性进行写操作（不需要 response）完成。
  d) 0x07 表示，对服务端属性进行读操作完成。
  e) 0x08 表示，对服务端的扫描服务操作完成。
  f) 0x09 表示，对服务端的扫描特征操作完成。
  g) 0x0A 表示，对服务端的扫描特征描述操作完成。

客户端 profile 回调函数参数的类型是 struct gatt_msg。解释如下:

```
struct gatt_msg {
    enum gatt_msg_evt msg_evt;  //!< The event that message come with
    uint8_t conn_idx;           //!< Connection index
    uint8_t svc_id;             //!< service id of this message
    uint16_t att_idx;           //!< Attribute index of in the service table or client uuid
table
    uint16_t handle;            //!< Attribute handle number in peer service
    union {
        struct {
            uint8_t operation;  //!< GATT request type
            uint8_t status;     //!< Status of the request
            uint16_t arg_len;   //!< Parameter length
            void * arg;         //!< Parameter pointer
        } gatt_op_cmp;          //!< Value of GATTC_MSG_CMP_EVT

        struct {
            uint16_t msg_len;   //!< GATT message length
            uint16_t offset;    //!< GATT message data offset
            uint8_t *p_msg_data;//!< GATT message data pointer
        } gatt_data;
    } param;
};
```

说明：

- msg_evt 元素是消息的类型。参见 gatt_msg_evt_t 的定义。客户端一共有以下几种
  a) GATTC_MSG_NTF_REQ，客户端接收到对端发送的 ntf 数据。
  b) GATTC_MSG_IND_REQ，客户端接收到对端发送的 indication 数据。
  c) GATTC_MSG_READ_IND，客户端接收到对端发送的 read 操作回复的数据。
  d) GATTC_MSG_CMP_EVT，profile 接收到某个事件完成的消息。
  e) GATTC_MSG_LINK_CREATE，profile 被通知有一个连接已经建立。
  f) GATTC_MSG_LINK_LOST，profile 被通知有一个连接已经断开。
  g) GATTC_MSG_SVC_REPORT，客户端接收到扫描到的服务信息。
  h) GATTC_MSG_CHAR_REPORT，客户端接收到扫描到的特征信息。
  i) GATTC_MSG_CHAR_DESC_REPORT，客户端接收到扫描到的特征描述信息。
- conn_idx 元素表示本消息来自于哪一个连接的连接号。
- svc_id，表示本消息对应的 profile 的 id 号，由 profile 添加函数返回。
- att_idx，表示本消息针对哪条属性，对客户端表示 uuid 数组的组号。对服务端表示 att 属性表的组号。
- handle，表示本消息针对的属性在服务端对应的 handle 号。
- param.gatt_data，表示在有数据相关的消息（除了 GATTC_MSG_CMP_EVT、GATTC_MSG_LINK_CREATE、GATTC_MSG_LINK_LOST）上传时的数据 buffer 指针和长度。

- param.gatt_op_cmp，表示消息类型为 GATTC_MSG_CMP_EVT 时，对应的操作号和完成的状态。注：对应操作为 GATT_OP_PEER_SVC_REGISTERED，param. gatt_op_cmp.arg 表示扫描到的属性的 handle 号，其他操作情况，无意义。

1.2.3.5 向服务端进行写操作

下面将描述客户端写服务端属性的详细步骤及注意事项。

1.标准流程

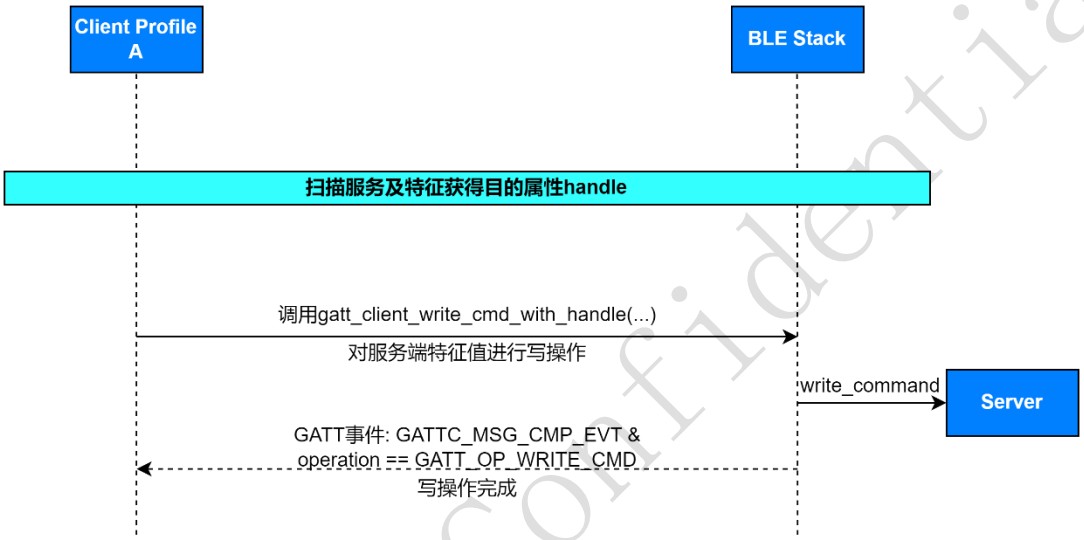


图 1-8 客户端 write_command 处理标准流程

说明：

- 在经过扫描特征后，应用层将得到对端特征值的 handle 以及许可属性，通过许可属性可判断该特征值是否可写，如果该特征值可写，则可以进行后续写操作。在扫描特征后，可调用 gatt_discovery_peer_desc 函数扫描服务端该特征拥有的特征描述，调用该函数将产生 GATTC_MSG_CHAR_DESC_REPORT 事件，事件会产生 N 次，N 取决于 handle 范围内特征描述的数量，该事件将上报特征描述的 uuid 以及 handle，用户亦可调用下述函数对特征描述进行写操作。
- 应用层可以调用 GATT 写函数对可写属性进行写操作。标准流程写函数定义在 gatt_api.h 内，一共有两种如下：
  a) gatt_client_write_req_with_handle 表示写操作，需要服务端回复 response。
  b) gatt_client_write_cmd_with_handle 表示写操作，不需要服务端回复 response。
- 调用 write_cmd 操作发送数据到协议栈，协议栈会分配一段内存缓存应用层发送的数据，然后立即上传写完成消息到客户端回调函数，消息是 GATTC_MSG_CMP_EVT & operation == GATT_OP_WRITE_REQ 或 GATT_OP_WRITE_CMD。用户可以在回调函数内判断此消息做为 write_cmd 数据发送完成处理。
- 应用层的数据在用户调用 write_req 时，操作完成的消息需要等到对端接收成功后才会上传。所以应用层在上一次操作完成前，对同一个属性只能调用一次 write_req，否则后面的 write_req 操作会无效。

Freqchip Confidential

- 应用层的数据在用户调用 write_cmd 发送数据(非 write_req)时，协议栈只是缓存数据到内部，在链路层可以发送的时候将缓存的数据发到对端 BLE 设备。所以应用层在往协议栈发送数据的时候，可以反复调用多次写操作，但要注意系统内存的消耗，一般建议设一个能往协议栈发送数据包的最个数，用一个计数器记录当前发包的个数，每发送完成一次计数器加一，每调写函数往下发一次，就减 1。示例代码如下：

```
uint8_t nb_report = 0;
uint16_t client_msg_handler(gatt_msg_t *p_msg)
{
    switch(p_msg->msg_evt)
    {
        case GATTC_MSG_CMP_EVT:
        {
            if(p_msg->param.gatt_op_cmp.operation == GATT_OP_WRITE_CMD )
            {
                nb_report--;
              write_att_cmd(p_msg->conn_idx,client_id,p_msg->param.gatt_data.p_msg_data
              ,p_msg->param.gatt_data.msg_len);
            }
        }
        break;
    }
}
void write_att_cmd(uint8_t conidx, uint8_t client_id, uint8_t *p_data, uint16_t data_len)
{
   if(nb_report < 20)
   {
      gatt_client_write_param_handle_t write;
      write.conidx     = conidx;
      write.client_id   = client_id;
      write.handle     = 1;  //
      write.p_data     = p_data;
      write.data_len   = data_len;
      gatt_client_write_cmd_with_handle(&write);
      nb_report++;
}
}
```

2.简化流程

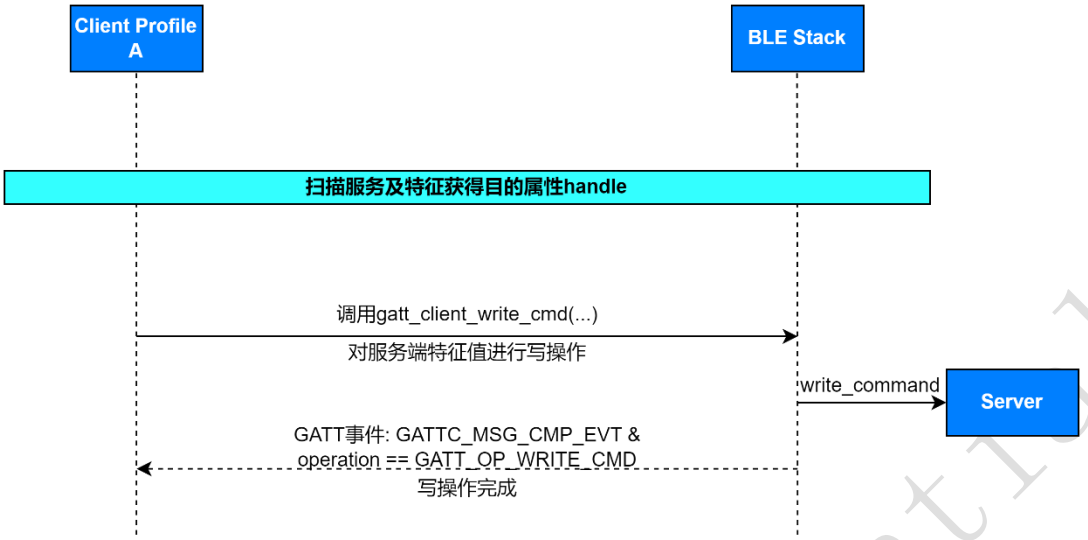


图 1-9 客户端 write_command 处理简化流程

说明：

- 在经过扫描服务操作后， 协议栈将自动保存关心 uuid 的 handle，并上报两次 GATT_MSG_CMP_EVT 事件，先后分别为扫描服务结束事件(GATT_OP_PEER_SVC_DISC_END)和保存 handle 的事件(GATT_OP_PEER_SVC_REGISTERED)，应用层随后可使用关心 uuid 数组的索引与服务端进行通信。
- 应用层可以调用 GATT 写函数对可写特征值进行写操作。简化流程写函数定义在 gatt_api.h 内，一共有两种如下：
  a) gatt_client_write_req 表示写操作，需要服务端回复 response。
  b) gatt_client_write_cmd 表示写操作，不需要服务端回复 response。
- 与标准流程的区别在于通信的方式由句柄改为了 uuid 数组的索引。
- 简化流程只能针对于服务端的特征值进行读写操作，即 uuid 数组里的 uuid 只能为特征值的 uuid。

示例代码如下：

```
uint16_t client_msg_handler(gatt_msg_t *p_msg)
{
    switch(p_msg->msg_evt)
    {
        case GATTC_MSG_CMP_EVT:
        {
            if(p_msg->param.gatt_op_cmp.operation == GATT_OP_WRITE_CMD )
            {
              uint8_t data[] = {0x02};
              gatt_client_write_param_t write;
              write.conidx      = p_msg->conn_idx;
              write.client_id   = client_id;
              write.att_idx     = 1;  //
              write.p_data      = data;
              write.data_len    = sizeof(data);
```

```
                gatt_client_write_cmd(&write);
            }
        }
        break;
    }
}
```

1.2.3.6 向服务端进行读操作

下面将描述客户端读服务端属性的详细步骤及注意事项。

1.标准流程

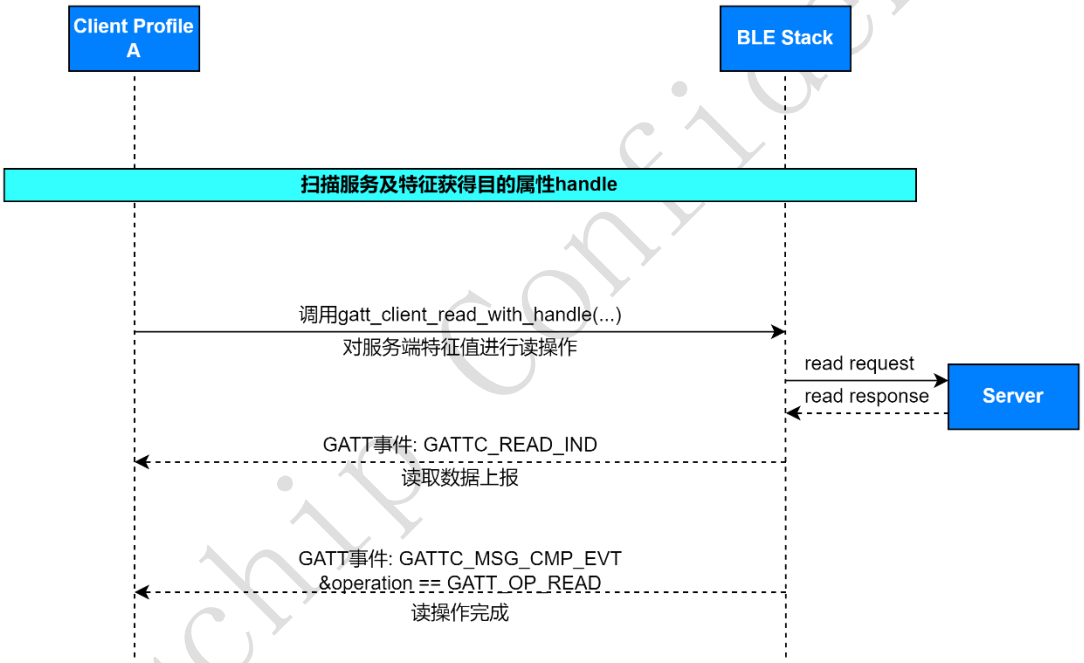


图 1-10 客户端 read 标准流程

说明：

- 在经过扫描特征后，应用层将得到对端特征值的 handle 以及许可属性，通过许可属性可判断该特征值是否可读，如果该特征值可读，则可以进行后续读操作。也可通过扫描特征描述获取特征描述 handle 后，进行读操作。
- 应用层可以调用 GATT 读函数对可读属性进行读操作。读函数为 gatt_client_read_with_handle，定义在 gatt_api.h 内。
- 调用 read 函数发起读操作，服务端回复 read_response 后会上传 GATT 事件 GATTC_MSG_READ_IND，该消息表示接收到对端回复的数据。
- READ_IND 事件上报之后，协议栈会再次上传消息 GATTC_MSG_CMP_EVT & operation == GATT_OP_READ，表示 read 操作完成。

Freqchip Confidential

2.简化流程

- 读取操作的简化流程与写操作相同，请参考写操作的简化流程。读取操作简化流程的读函数为 gatt_client_read，定义在 gatt_api.h 内。
- 简化流程只能针对于服务端的特征值进行读写操作，即 uuid 数组里的 uuid 只能为特征值的 uuid。

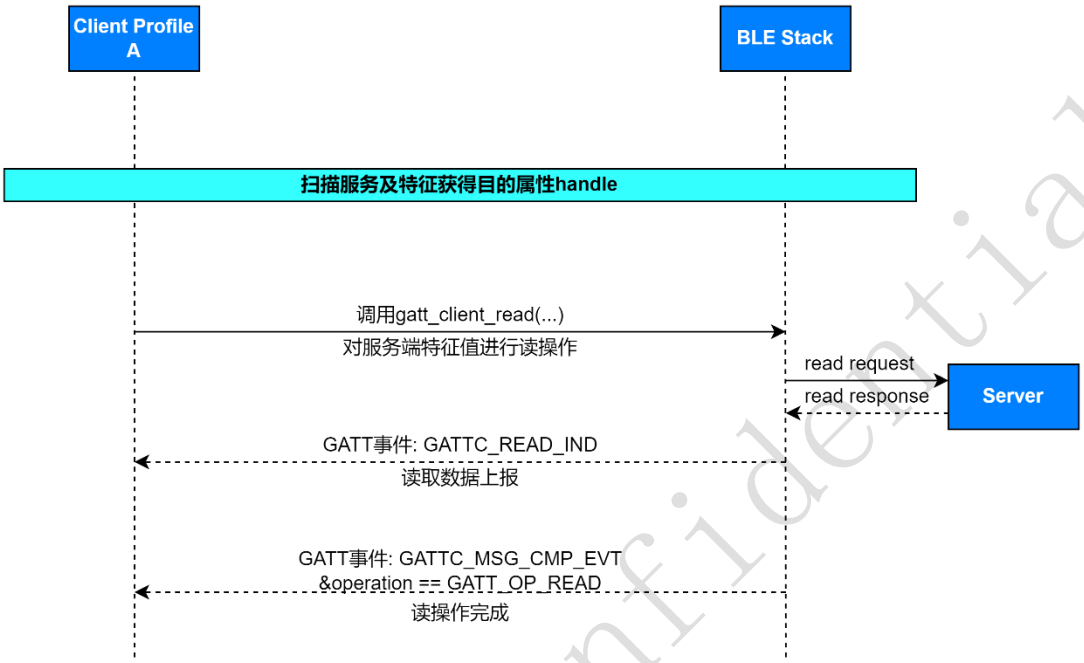


图 1-11 客户端 read 简化流程

1.2.3.7 客户端使能 ntf/ind

1.标准流程

若客户端需要使能 notification 和 indication，则可以在扫描特征后，根据 GATTC_MSG_CHAR_REPORT 事件上报的特征许可属性 prop 判断该特征是否具有 ntf 或者 ind 这个 prop。如果该特征具有这个 prop，则客户端可以调用 gatt_discovery_peer_desc 扫描该特征的特征描述判断该特征是否具有 uuid 为 0x2902 的特征描述，若含有则客户端可以通过对该特征描述进行写操作来使能 ntf 或者 ind，对该特征写入数据 0x0100 表示使能 ntf，写入 0x0200 表示使能 ind。流程图如下。

FreqChip Confidential

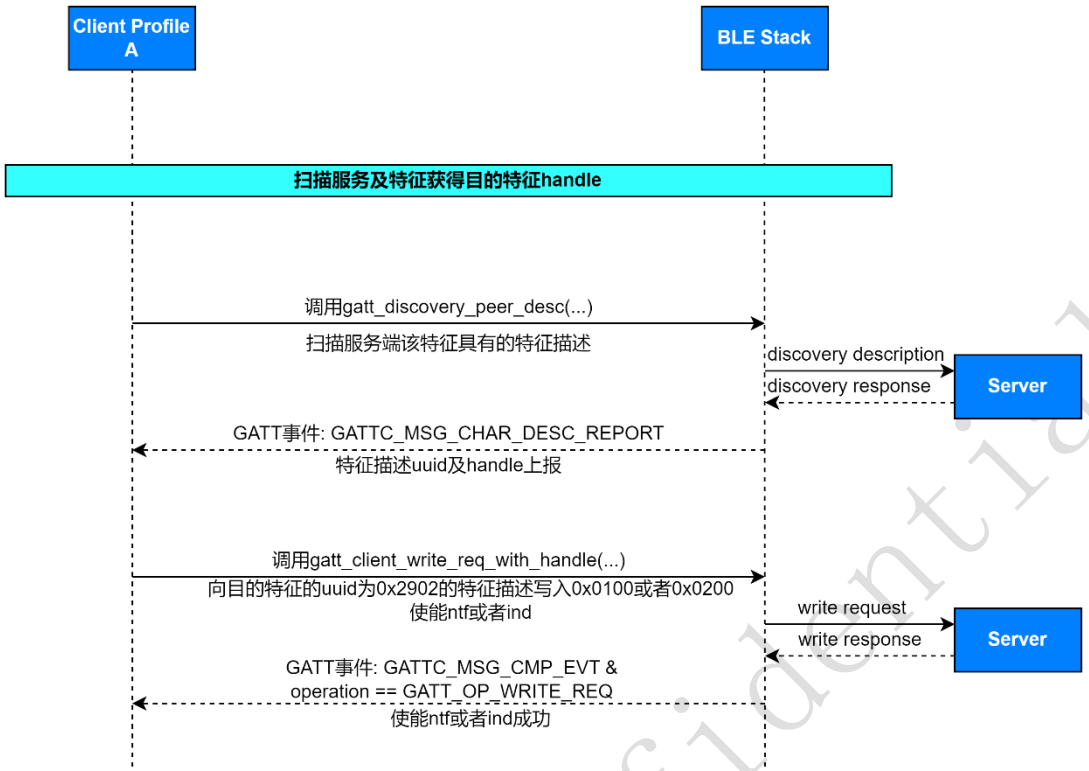


图 1-12 客户端使能 ntf、ind 流程(标准流程)

2.简化流程

对于简化流程，协议栈会在扫描对端服务时，自动判断关注的特征是否含有 uuid 为 0x2902 的特征描述，若含有则客户端会自动保存该特征描述的 handle。因此用户如果需要使能 ntf/ind，调用 gatt_api.h 中的函数 gatt_client_enable_ntf_ind 即可。流程图如下。

Freqchip Confidential

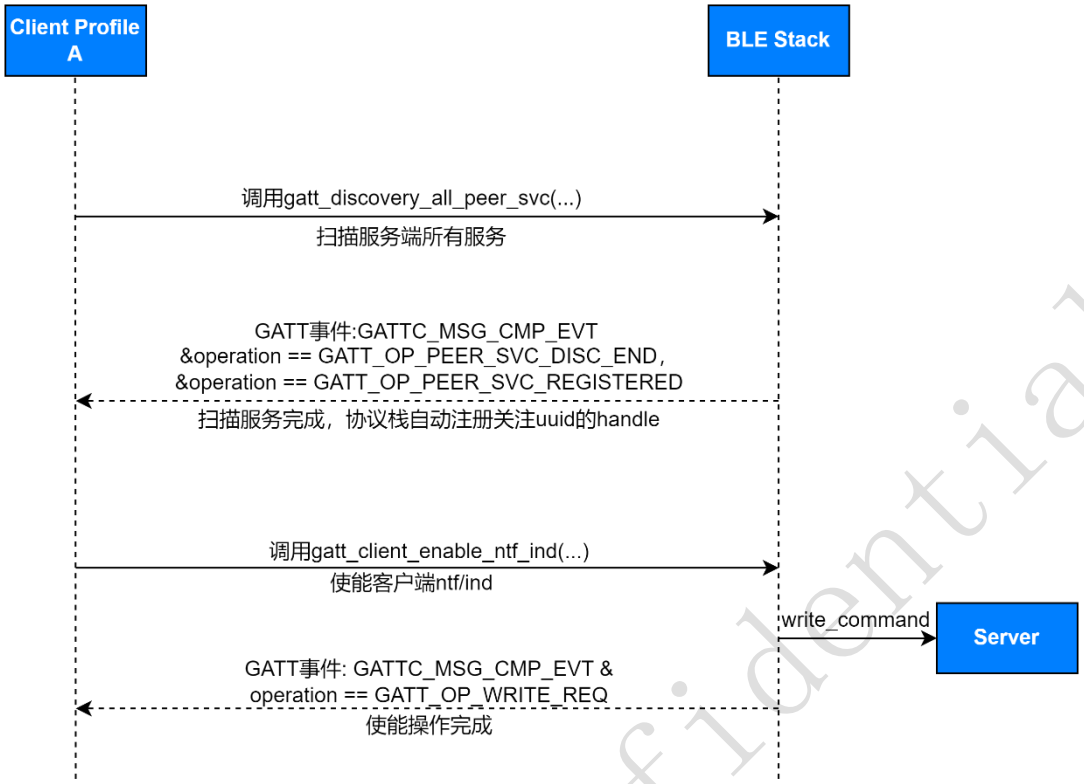


图 1-13 客户端使能 ntf、ind 流程(简化流程)

1.2.4 GATT 服务端基础流程

1.2.4.1 服务端

GATT 服务端负责接收对端设备 GATT 客户端发送的命令以及请求，并且根据收到的命令以及请求回复对端设备响应，或者向对端设备发送通知或指示。

下面章节将描述如何在 GATT 服务端添加服务和注册服务回调函数，以及 GATT 服务端如何处理客户端的读/写请求。关于 API 的详细描述，请参考 gatt_api.h 内函数定义说明。

1.2.4.2 注册服务

针对服务端，用户需给出一个属性表 att_table 和服务的回调函数并通过 gatt_add_service 添加进协议栈，便于与客户端的交互。流程图如下。

Freqchip Confidential

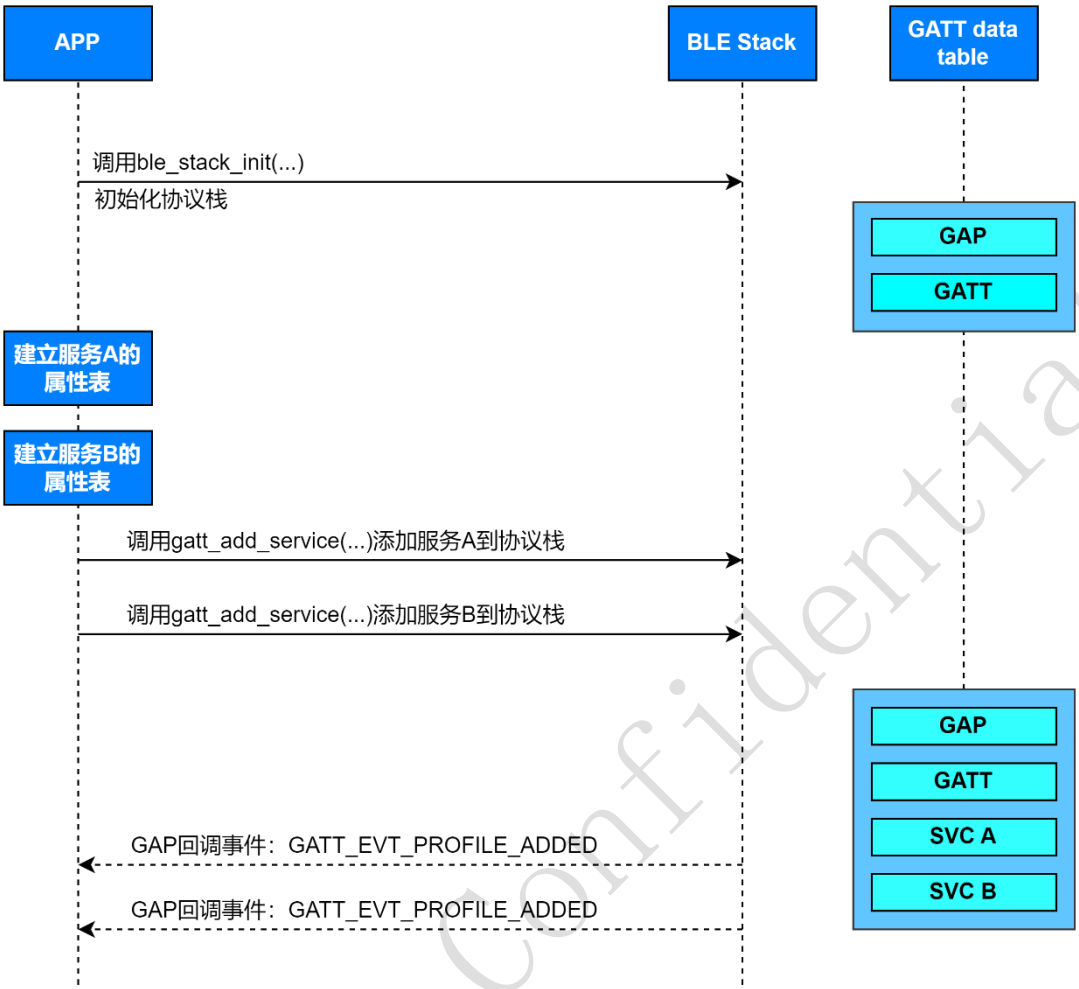


图 1-14 服务端添加服务流程

以一个添加服务的实例来解释该过程。

```
uint8_t svc_id = 0;
const uint8_t svc_group_uuid[] =
"\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f";
enum
{
    SP_IDX_SERVICE,
    SP_IDX_CHAR1_DECLARATION,
    SP_IDX_CHAR1_VALUE,
    SP_IDX_CHAR1_CFG,
    SP_IDX_CHAR1_USER_DESCRIPTION,
    SP_IDX_CHAR2_DECLARATION,
    SP_IDX_CHAR2_VALUE,
    SP_IDX_CHAR2_USER_DESCRIPTION,
    SP_IDX_NB,
```

Freqchip Confidential

```
};
const gatt_attribute_t att_tb[] =
{
    [SP_IDX_SERVICE] = { {UUID_SIZE_2,UUID16_ARR(GATT_PRIMARY_SERVICE_UUID)}
        ,GATT_PROP_READ,UUID_SIZE_16,(uint8_t *)svc_group_uuid
    },

    [SP_IDX_CHAR1_DECLARATION] = { {UUID_SIZE_2,UUID16_ARR(GATT_CHARACTER_UUID)}
        ,GATT_PROP_READ,0,NULL
    },
    [SP_IDX_CHAR1_VALUE] = { {UUID_SIZE_16,  {0x00, 0x00, 0x48, 0x43, 0x45, 0x54, 0x43, 0x49,
0x47, 0x4F, 0x4C, 0x49, 0x04, 0x18, 0x00, 0x00,}}
        ,GATT_PROP_READ|GATT_PROP_NOTI|GATT_PROP_INDI,20,NULL
    },
    [SP_IDX_CHAR1_CFG] = { {UUID_SIZE_2,UUID16_ARR(GATT_CLIENT_CHAR_CFG_UUID)}
        ,GATT_PROP_READ|GATT_PROP_WRITE,0,NULL
    },
    [SP_IDX_CHAR1_USER_DESCRIPTION] = { {UUID_SIZE_2,UUID16_ARR(GATT_CHAR_USER_DESC_UUID)}
        ,GATT_PROP_READ|GATT_PROP_WRITE,12,NULL
    },
};
uint16_t svc_msg_handler(gatt_msg_t *p_msg)
{
    switch(p_msg->msg_evt)
    {
        case GATTS_MSG_READ_REQ:
        break;
     }
}
void user_main(void)
{
    gatt_service_t service;
    service.p_att_tb = att_tb;
    service.att_nb = SP_IDX_NB;
    service.gatt_msg_handler = svc_msg_handler;
    svc_id=gatt_add_service(&service);
}
```

说明：

- att_tb[]是一个属性表，属性表定义参见 1.2.2.4。
- svc_id 变量记录这个服务端的服务 id 号，后续对客户端进行 ntf/ind 操作时需要用到这个数值。
- svc_msg_handler，定义的是服务端 service 的事件回调函数，协议栈需要上报 GATT 事件时，会调用该函数。每个 service 可含有各自的回调函数。
- 定义了服务端 service 的变量，并将属性表数组和回调函数赋值给该变量后，调用 gatt_add_service 函数将该服务端 service 添加到协议栈内部去。
- 用户可以通过 gatt_add_service 函数反复添加多个服务端 service 到协议栈内部。
- 添加 service 成功后，系统会上报 GAP 事件 GATT_EVT_PROFILE_ADDED。

1.2.4.3 删除服务

用户在添加完服务后，可调用 gatt_delete_user_service 函数删除已添加的服务。

1.2.4.4 服务端 service 回调函数

为了处理客户端与服务端的通信操作，服务端需要定义自己的读写回调函数。回调函数接收协议栈上传的对应 service 的所有事件。以一个服务端 service 的回调函数实例来解释。

```
uint16_t svc_msg_handler(struct gatt_msg *p_msg)
{
    switch(p_msg->msg_evt)
    {
        case GATTS_MSG_READ_REQ:
            if(p_msg->att_idx == SP_IDX_CHAR1_VALUE)
            {
                memcpy(p_msg->param.gatt_data.p_msg_data, "\x7\x8\x9", 3);
                return 3;
            }
            break;
        case GATTS_MSG_WRITE_REQ:
            if(p_msg->att_idx == SP_IDX_CHAR1_VALUE)
                show_reg(p_msg->param.gatt_data.p_msg_data,p_msg->param.gatt_data.msg_len,1);
            break;
        case GATTC_MSG_CMP_EVT:
            if(p_msg->param.gatt_op_cmp.operation == GATT_OP_NOTIFY)
                co_printf("ntf done\r\n");
            break;
        case GATTC_MSG_LINK_CREATE:
            co_printf("link create\r\n");
            break;
```

```
        case GATTC_MSG_LINK_LOST:
            co_printf("link lost\r\n");
            break;
        default:
            break;
    }
    return 0;
};
```

说明：

- GATT 事件回调函数会传入一个消息变量。该变量包含了服务 id、事件的类型、连接号、服务 id 对应的属性表的序号、事件对应的 handle 号、事件的内容。
- GATT 的事件回调函数根据上传的事件的类型分别处理，函数以 return 0 结尾。
- 服务端的事件类型主要有以下 3 种：
  a) GATTS_MSG_READ_REQ，服务端接收到对端的读操作，该事件只会在对应属性数据部分的 p_data 指针为 NULL 时上报。
  b) GATTS_MSG_WRITE_REQ，服务端接收到对端的写操作（write_req 和 write_cmd）。
  c) GATTS_MSG_ATT_INFO_REQ，服务端接收到对端获取属性信息请求。
- 当上报事件是 GATTC_MSG_READ_REQ 时，若读请求对应属性数据部分的 p_data 指针为 NULL 时，需要对消息变量中的 p_msg_data 指针赋值达到回复读请求的目的，示例中做了回复，注意：回复完毕之后，需要 return 回复数据的长度，而不是 return 0；若 p_data 指针有值时，则协议栈将从中取值自动回复给客户端。
- 消息是 GATTS_MSG_CMP_EVT 表示针对服务端 service 的某个操作完成。
  p_msg->param.gatt_op_cmp.operation 代表操作的类型，类型一共有以下几种，参见在 gatt_api.h 内宏定义组 GATT_OPERATION_NAME。

```
#define GATT_OP_NOTIFY                  0x01    //!< GATT notification operation
#define GATT_OP_INDICA                  0x02    //!< GATT indication operation
#define GATT_OP_PEER_SVC_REGISTERED     0x03    //!< Used with GATTC_CMP_EVT, GATT peer device
                                                //!< service registered
#define GATT_OP_WRITE_REQ               0x05    //!< GATT wirte request operation
#define GATT_OP_WRITE_CMD               0x06    //!< GATT wirte command operation, write
                                                //!< without response
#define GATT_OP_READ                    0x07    //!< GATT read operation
#define GATT_OP_PEER_SVC_DISC_END       0x08    //!< Used with GATTC_CMP_EVT, GATT peer device
                                                //!< service discovery is ended
#define GATT_OP_PEER_CHAR_DISC_END      0x09    //!< Used with GATTC_CMP_EVT, GATT peer device
                                                //!< characteristic discovery is ended
#define GATT_OP_PEER_DESC_DISC_END      0x0A   //!< Used with GATTC_CMP_EVT, GATT peer device
                                //!< characteristic describtion discovery is //!< ended
```

对服务端 profile 来说，能用到的操作有，0x01、0x02。

- 0x01 表示，对客户端属性进行 notification 操作（不需要 response）完成。
- 0x02 表示，对客户端属性进行 indication 操作（需要 response）完成。

服务端 profile 回调函数参数的类型是 struct gatt_msg。与客户端 profile 输入参数的解释相同。参见章节 1.2.3.4。

1.2.4.5 向客户端进行 ntf/ind 操作

向客户端进行 ntf/ind 操作的交互流程如下图所示。

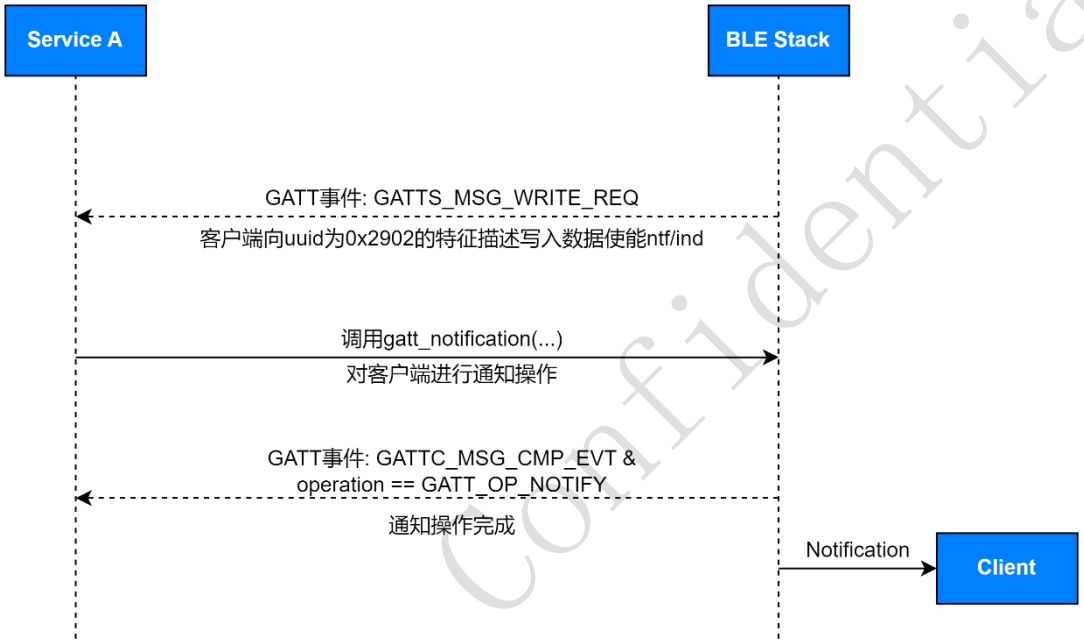


图 1-15 服务端 notification 处理流程

说明：

- 在收到协议栈上传的 GATT 事件 GATTS_MSG_WRITE_REQ 后，判断写入数据的第一个字节 bit 0 置位，确定 ntf 使能。判断写入数据的第一个字节 bit 1 置位，确定 ind 使能。
- Ntf 使能后，应用层可以调用 GATT 写函数对被使能 ntf 的属性进行通知操作。通知函数定义在 gatt_api.h 内，一共有两种如下：
  a) gatt_notification 表示通知操作，不需要服务端回复 response。
  b) gatt_indication 表示指示操作，需要服务端回复 response。
- 调用 notification 操作发送数据到协议栈，协议栈会分配一段内存缓存应用层发送的数据，然后立即上传 ntf 完成消息到服务端回调函数，消息是 GATTC_MSG_CMP_EVT & operation == GATT_OP_NOTIFY。用户可以在回调函数内判断此消息做为 notification 数据发送完成处理。
- 调用 indication 操作发送数据到协议栈，操作完成的消息需要等到对端接收成功后才会上传。所以应用层在上一次操作完成前，对同一个属性只能调用一次 indication 操作，否则后面的 indication 操作会无效。
- 应用层的数据在用户调用 notification 发送数据时，协议栈只是缓存数据到内部，在链路层可以发送的时候将缓存的数据发到对端 BLE 设备。所以应用层在往协议栈发送数据的时候，可以反复调用多次写操作，但

freqchip Confidential

要注意系统内存的消耗，一般建议设一个能往协议栈发送数据包的最个数，用一个计数器记录当前发包的个数，每发送完成一次计数器加一，每调写函数往下发一次，就减 1。示例代码如下：

```
uint8_t nb_report = 0;
uint16_t svc_msg_handler (struct gatt_msg *p_msg)
{
    switch(p_msg->msg_evt)
    {
        case GATTC_MSG_CMP_EVT:
        {
            if(p_msg->param.gatt_op_cmp.operation == GATT_OP_NOTIFY )
            {
                 nb_report--;
                 att_ntf(p_msg->conn_idx,svc_id,p_data, data_len );
            }
        }
        break;
    }
}
void att_ntf( uint8_t conidx, uint8_t svc_id,uint8_t *p_data, uint16_t data_len )
{
   if(nb_report < 20 && ntf_enable[condix])
   {
      gatt_send_event_t ntf_att;
      ntf_att.conidx = conidx;
      ntf_att.svc_id = svc_id;
      ntf_att.att_idx = 2;
      ntf_att.p_data = p_data;
      ntf_att.data_len =data_len;
      gatt_notification (ntf_att);
      nb_report++;
    }
}
```

1.2.4.6 向客户端回应读操作

服务端接收到读请求后，向客户端回复数据的交互流程如下图所示。该回复步骤只针对于当被读取属性的数据指针 p_data 为 NULL 的情况。若 p_data 不为 NULL，则协议栈将自动从 p_data 获取最大可传输数据长度的数据回复给客户端。

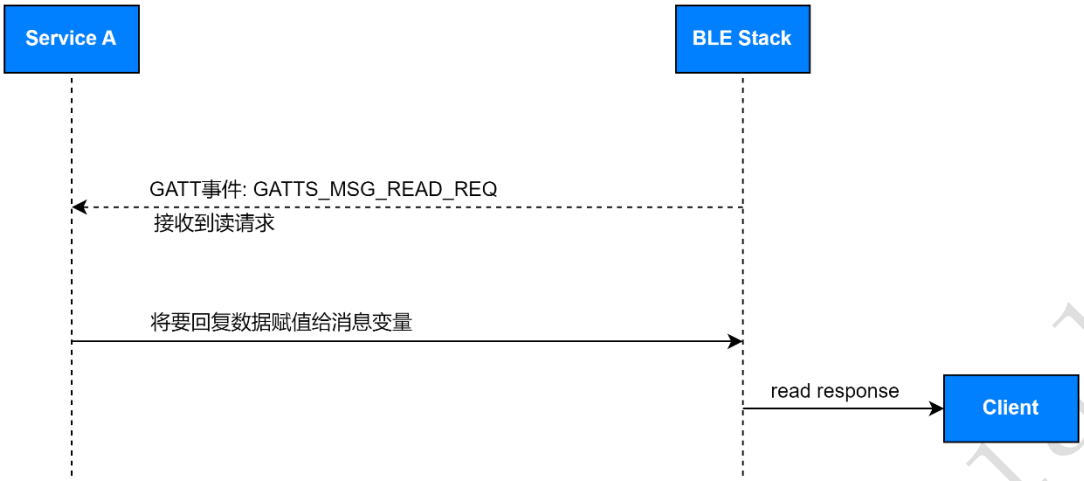


图 1-16 服务端 read response 处理流程

说明：

- 收到协议栈上传的 GATT 消息 GATTC_MSG_READ_REQ，表明客户端需要读某个属性的数据，服务端需要对这个读操作进行回复，如下代码。

```
uint16_t svc_msg_handler(gatt_msg_t *p_msg)
{
    switch(p_msg->msg_evt)
    {
        case GATTS_MSG_READ_REQ:
            if(p_msg->att_idx == 2)
            {
                memcpy(p_msg->param.gatt_data.p_msg_data, "\x7\x8\x9", 3);
                return 3;
            }
        break;
    }
}
```

首先判断读消息针对的属性在属性表中的序号是 2，然后将要回复的读数据直接拷贝到回调函数参数的数据指针指向的 buffer，最后返回拷贝数据的长度即可。回调函数返回后，协议栈会发送 read reponse 包给客户端。

1.3 SMP

1.3.1 配对

配对过程，即生成和分发密钥的过程，可以分为 3 个阶段：

1. 交换配对信息。

2. 生成链路加密密钥。
3. 在加密链路上分发其它指定的密钥信息，并根据设备是否支持绑定决定是否需要在安全数据库中存储分发的密钥信息。

配对流程图如下所示，配对方式分成 LE Legacy Pairing 和 LE Secure Connection（从 core 4.2 版本添加进来）两种，二者区别主要在于 step2-生成密钥的过程，在该步骤，LE Legacy Pairing 生成 STK，LE Secure Connection 生成 LTK。

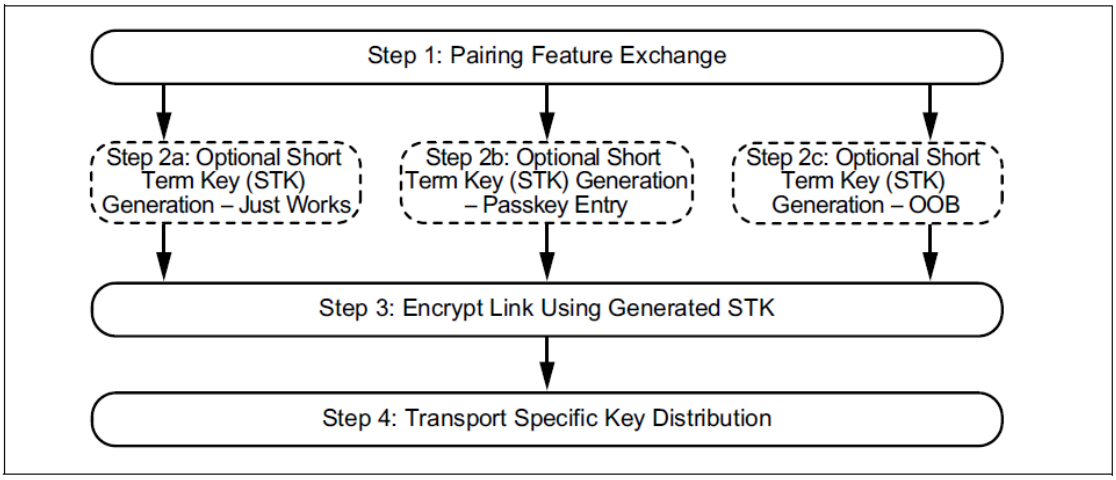


图 1-17 BLE 配对过程

LE Secure Connection 中引入了 Elliptic Curve Diffie-Hellman 加密算法，使得安全性得到了很大的提升。根据设备所支持的安全特性，根据 IO 能力配对方法可分为如下四种：

- Just Works（Secure Connections or Legacy），适合于配对双方没有输入输出 io 设备（比如：键盘输入或显示器输出）的情况，由于在 Just Works 配对过程中无需 MITM 认证，因此也就无法抵御 MITM 攻击。Just Works 配对既可以用于 Legacy Pairing，也可以用于 Secure Connection。
- Passkey Entry（Secure Connections or Legacy），配对流程支持 MITM 认证，既可用于 Legacy Pairing，也可以用于 Secure Connection。配对过程中需要用户输入 6 位十进制数的密码，作为配对过程所需的输入参数。
- Numeric Comparison（Secure Connections），如果双方设备都支持 Secure Connection，IO 能力都具有显示和输入功能，且需带 MITM 认证，则可以使用 Numeric Comparison 配对流程。
- Out of Band（Secure Connections or Legacy），该方式通过 NFC 等带外方式交互配对过程需要的中间值。目前版本的 SDK 中未支持该配对方式。

1.3.1.1 JustWorks 配对

如果双方设备都无需带中间人（Man-in-the-middle，MITM）认证，那么就可以使用 Just Works 配对流程。由于在 Just Works 配对过程中无需 MITM 认证，因此也就无法抵御 MITM 攻击。Just Works 配对既可以是 Legacy Pairing 也可以是 Secure Connection，用户只需在配对流程开始之前配置好安全参数，配对流程启动后，配对过程中无需用户的任何交互。

下面以 Legacy Pairing、Justworks 配对方式为例，介绍参数如何配置：

```
gap_security_param_t param =
{
   .mitm = false,
   .secure_connection = false,
   .bond = true,
   .rsp_mode = ENABLE_AUTO_RSP,
   .oob_used = GAP_OOB_AUTH_DATA_NOT_PRESENT,
   .io_cap = GAP_IO_CAP_NO_INPUT_NO_OUTPUT,
};
gap_security_param_init(&param);
```

说明：

- mitm 参数表示是否启动带中间人的认证，设为 false 表示不启用。
- secure_connection 参数表示是否启动 security connection。设为 false 表示不启用。
- bond 参数表示是否启用绑定，也就是配对完成之后是否保存分发的密钥，用于下次的连接。设为 true 表示启用绑定。
- rsp_mode 参数表示安全响应模式，ENABLE_AUTO_RSP 表示会自动回复配对或加密请求。
- oob_used 参数表示是否存在 oob 带外数据，本示例表示不存在。
- io_cap 表示本设备的 io 输出能力，是否有显示或键盘输入的能力。本示例中没有 io 能力。

1.3.1.2 Passkey Entry 配对

Passkey Entry 配对流程支持 MITM 认证，配对既可以是 Legacy Pairing 也可以是 Secure Connection。配对过程中拥有 display 输出能力的一方显示 6 位十进制密码，拥有 keyboard 输入能力的一方输入该密码来作为 temp key，进行后续的配对流程。

下面以具有输出能力的一种配置介绍 Passkey entry 配对模式的参数设置：

```
gap_security_param_t param =
{
   .mitm = true,
   .secure_connection = false,
   .bond = true,
  .rsp_mode = ENABLE_AUTO_RSP,
   .oob_used = GAP_OOB_AUTH_DATA_NOT_PRESENT,
   .io_cap = GAP_IO_CAP_DISPLAY_ONLY,
};
gap_security_param_init(&param);
```

说明：

- mitm 参数表示是否启动带中间人的认证，设为 true 表示启用。
- secure_connection 参数表示是否启动 security connection。设为 false 表示不启用。

- bond 参数表示是否启用绑定，也就是配对完成之后是否保存分发的密钥，用于下次的连接。设为 true 表示启用绑定。
- rsp_mode 参数表示安全响应模式，ENABLE_AUTO_RSP 表示会自动回复配对或加密请求。
- oob_used 参数表示是否存在 oob 带外数据，本示例表示不存在。
- io_cap 表示本设备的 io 输出能力，是否有显示或键盘输入的能力。本示例中没有输入输出能力。

当本地设备需要输入 tk 密钥或者显示 tk 密钥时，协议栈会上传 GAP 事件 GAP_EVT_SMP_TK_REQ，此时可调用如下函数使协议栈完成配对后续操作：

```
ble_api_err_t gap_security_tk_rsp(uint8_t conidx, uint32_t tk, bool accept);
```

说明：

- conidx：该连接的索引。
- tk：密钥，取值范围为 100000 到 999999，ble 的 tk 必须为 6 位。
- accept：参数为 true 表示同意配对，false 表示拒绝配对
- 上报该事件时，本地设备在交换 tk 过程中扮演的角色(keyboard or display)也会上报至 GAP 回调函数的消息变量中。

1.3.1.3 Numeric comparison 配对

Numeric comparison 只能使用 Secure Connection，而 Secure Connection 需要双方设备都支持 Secure Connection 配对。具体的安全参数配置可参考如下：

```
gap_security_param_t param =
{
   .mitm = false,
   .secure_connection = true,
   .bond = true,
  .rsp_mode = ENABLE_AUTO_RSP,
   .oob_used = GAP_OOB_AUTH_DATA_NOT_PRESENT,
   .io_cap = GAP_IO_CAP_KB_DISPLAY,
};
gap_security_param_init(&param);
```

说明：

- mitm 参数表示是否启动带中间人的认证，设为 false 表示不启用。
- secure_connection 参数表示是否启动 security connection。设为 true 表示启用。
- bond 参数表示是否启用认证。设为 true 表示启用认证。
- rsp_mode 参数表示安全响应模式，ENABLE_AUTO_RSP 表示会自动回复配对或加密请求。
- oob_used 参数表示是否存在 oob 带外数据，本示例表示不存在。
- io_cap 表示本设备 io 能力，是否有显示或键盘输入的能力。本示例中没有 io 能力。

当使用 Numeric comparison 配对时，本地设备需要通过数字比较来进行确认，协议栈会上传 GAP 事件 GAP_EVT_SMP_NC_REQ，此时可调用 gap_security_nc_rsp 使协议栈完成配对后续操作：

Freqchip Confidential

```
case GAP_EVT_SMP_NC_REQ :
    printf("gap_callback: conidx: %d, GAP_EVT_SMP_NC_REQ\r\n",event->param.nc_req.conidx);
    printf("number is %d\r\n",event->param.nc_req.number);
    gap_security_nc_rsp(event->param.nc_req.conidx, true);
  break;
```

1.3.1.4 主机发起配对

主机发起配对操作后，应用层与 BLE 协议栈的交互过程如下图所示。

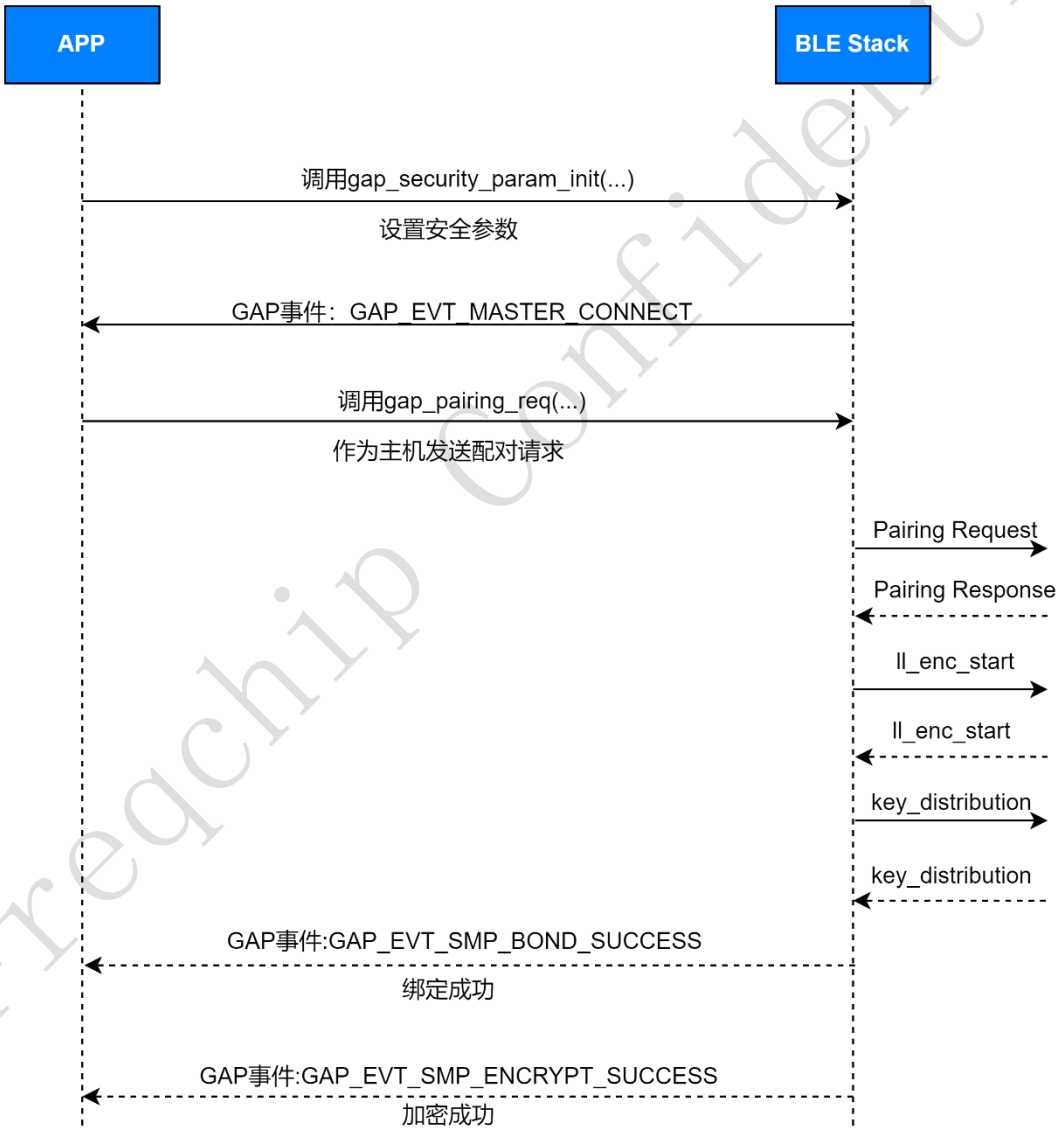


图 1-18 主机发起配对操作

主机开启配对操作流程的具体步骤如下：

```
void proj_ble_gap_evt_func(gap_event_t *event)
```

Freqchip Confidential

```
{
    switch(event->type)
    {
        case GAP_EVT_MASTER_CONNECT:
      if(!gap_get_bond_status(event->param.connect.conidx)) {
                    /* If it has not been bonded to the peer device, send a bond request */
                   printf("not bond with peer device, conidx:%d\r\n",
      event->param.connect.conidx);
                   gap_pairing_req(event->param.connect.conidx);
            } else
                printf("already bonded\r\n");
        break;

    case GAP_EVT_SMP_SEC_REQ:
        if(!gap_get_bond_status(event->param.connect.conidx)) {
                /* If it has not been bonded to the peer device, send a bond request */
                 printf("not bond with peer device, conidx:%d\r\n",
event->param.connect.conidx);
                 gap_pairing_req(event->param.connect.conidx);
         }else
              printf("already bonded\r\n");
    break;

        case GAP_EVT_SMP_BOND_SUCCESS:
                printf("gap_callback: conidx: %d, GAP_EVT_SMP_BOND_SUCCESS\r\n",
           event->param.bond.conidx);
        break;
     }
}
```

说明：

- 在 GAP 事件回调函数内，通过判断事件类型为 GAP_EVT_MASTER_CONNECT 来获取主机连接事件处理的入口。在该事件下面调用 gap_get_bond_status()判断当前链接的对端设备是否是已绑定设备，如果是未绑定，调用 gap_pairing_req()函数发起配对动作。
- 当从机发起 securtity request，主机会上报 GAP_EVT_SMP_SEC_REQ 事件，在该事件中调用相关 api 来发起配对动作。
- BLE 协议栈上传 GAP 事件 GAP_EVT_SMP_BOND_SUCCESS，说明绑定成功。

1.3.1.5 从机发起配对

从机通过调用 gap_security_req 来请求主机发起配对操作，应用层与 BLE 协议栈的交互过程如下图所示

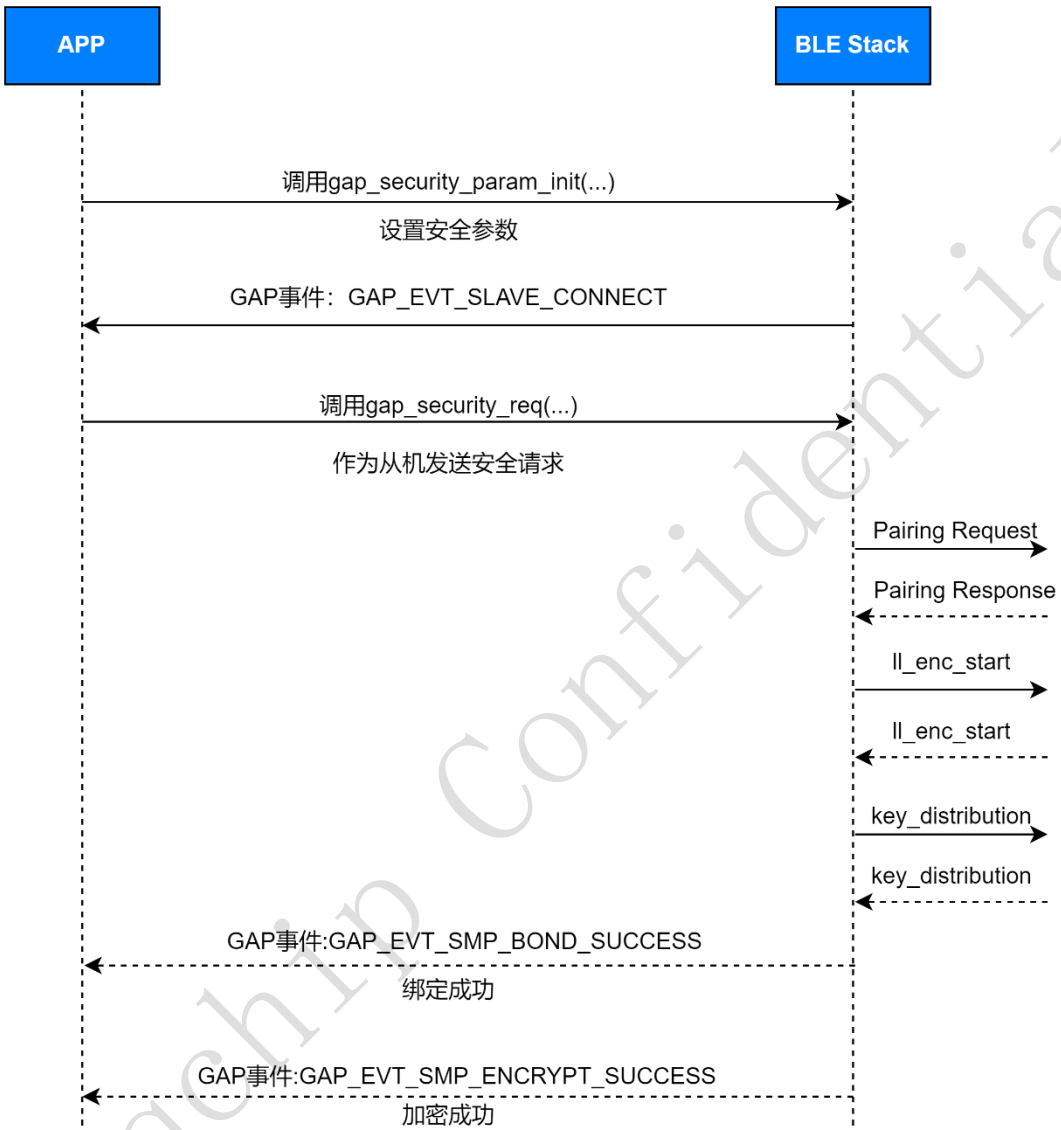


图 1-19 从机发起配对操作

从机请求配对流程的具体步骤如下：

```
void proj_ble_gap_evt_func(gap_event_t *event)
{
    switch(event->type)
    {
        case GAP_EVT_SLAVE_CONNECT:
                gap_security_req(event->param.connect.conidx);
        break;

        case GAP_EVT_SMP_BOND_SUCCESS:
```

Freqchip Confidential

```
            printf("bond condix:%d\r\n", event->param.bond.conidx);
        break;
}
```

说明：

- 在 GAP 事件回调函数内，通过判断事件类型为 GAP_EVT_SLAVE_CONNECT 来获取从机连接事件处理的入口。在该事件下面调用 gap_security_req 函数请求主机发起绑定动作。
- BLE 协议栈上传 GAP 事件 GAP_EVT_SMP_BOND_SUCCESS，说明绑定成功。

当 rsp_mode 参数设为 DISABLE_AUTO_RSP 时，主机发起的配对或者加密请求会上报至 GAP 回调函数，在 GAP_EVT_SMP_PAIR_REQ 或者 GAP_EVT_SMP_ENCRYPT_REQ 事件中，通过 gap_pairing_rsp 和 gap_encrypt_rsp 函数由用户自行决定是否配对和加密。

```
void proj_ble_gap_evt_func(gap_event_t *event)
{
    switch(event->type)
    {
     case GAP_EVT_SMP_PAIR_REQ :
        printf("gap_callback: conidx: %d,GAP_EVT_SMP_PAIR_REQ\r\n",
              event->param.pair_req.conidx);
        gap_pairing_rsp(event->param.pair_req.conidx,true);
        break;

     case GAP_EVT_SMP_ENCRYPT_REQ:
        printf("gap_callback: conidx: %d,GAP_EVT_SMP_ENCRYPT_REQ\r\n",
              event->param. encrypt_req.conidx);
          gap_encrypt_rsp(event->param.encrypt_req.conidx,event->param.encrypt_req.ltk,true);
        break;
  }
}
```

1.3.2 加密

设备进行做配对之后，只要开启了绑定管理功能。可以直接调用加密函数对已建立的链接进行加密操作。加密操作是主机在链接建立后发起的。

1.3.2.1 主机发起加密

```
void proj_ble_gap_evt_func(gap_event_t *event)
{
```

```
    switch(event->type)
    {
        case GAP_EVT_MASTER_CONNECT:
               if(!gap_get_bond_status(event->param.connect.conidx)) {
                     /* If it has not been bonded to the peer device, send a bond request */
                    gap_pairing_req(event->param.connect.conidx);
               }
            else {
            /* If it has already been bonded to the peer device, send an encryption request */
                    gap_encrypt_req(event->param.connect.conidx);
            }
        break;

        case GAP_EVT_SMP_BOND_SUCCESS:
              gap_encrypt_req(event->param.connect.conidx);
        break;

       case GAP_EVT_SMP_ENCRYPT_SUCCESS:
             printf("gap_callback: conidx: %d, GAP_EVT_SMP_ENCRYPT_SUCCESS\r\n",
                 event->param.encrypt.conidx);
        break;
  }
}
```

说明：

- 在 GAP 事件回调函数内，通过判断事件类型为 GAP_EVT_MASTER_CONNECT 来获取主机连接事件处理的入口。在该事件下面调用 gap_get_bond_status()判断当前链接的对端设备是否是已绑定设备，如果未绑定，先调用 gap_pairing_req()进行配对绑定，绑定成功后进入 GAP_EVT_SMP_BOND_SUCCESS 事件，再调用 gap_encrypt_req()函数发起加密。如果是已绑定设备，直接调用 gap_encrypt_req()函数发起加密。
- 加密成功后，BLE 协议栈会上传 GAP 事件 GAP_EVT_SMP_ENCRYPT_SUCCESS。

1.3.2.2 从机响应加密

```
void proj_ble_gap_evt_func(gap_event_t *event)
{
    switch(event->type)
    {
        case GAP_EVT_SLAVE_CONNECT:
        {
              gap_security_req(event->param.connect.conidx);
        }
```

```
        break;
        case GAP_EVT_SMP_BOND_SUCCESS:
              printf("bond condix:%d\r\n", event->param.bond.conidx);
        break;
       case GAP_EVT_SMP_ENCRYPT_SUCCESS:
              printf("encrypt condix:%d\r\n", event->param.encrypt.conidx);
        break;
  }
}
```

说明：

- 在 GAP 事件回调函数内，通过判断事件类型为 GAP_EVT_SLAVE_CONNECT 来获取从机连接事件处理的入口。
- 加密成功后，BLE 协议栈会上传 GAP 事件 GAP_EVT_SMP_ENCRYPT_SUCCESS。

1.3.3 隐私管理

BLE 中的隐私管理，是指已认证设备可以跟踪识别目标设备，而其他非认证设备无法跟踪识别目标设备。隐私管理使得已认证的设备可以正常地与目标设备进行连接和通讯，同时防止其他非认证设备、恶意破坏设备对目标设备的跟踪。

1.3.3.1 开启隐私管理功能

Ble 的隐私机制有白名单与解析名单，通过设置白名单可以只允许特定的蓝牙设备进行交互，解析名单通过保存双方设备的地址和密钥来提高安全性。

系统初始化时可以设置白名单，可以在广播和扫描时过滤掉白名单以外的设备。开启白名单管理功能的具体步骤如下：

1. 主设备与从设备进行连接并绑定，绑定过程中需要交换 IRK 以及身份地址信息，绑定之后主设备与从设备断开连接。。
2. 白名单配置。

```
oid proj_ble_gap_evt_func(gap_event_t *event)
{
 switch(event->type)
 {
    case GAP_EVT_SMP_BOND_SUCCESS:
        {
            if( (peer_device.addr_type == 0x01) && ((peer_device.addr.addr[5] & 0xC0) != 0xC0))
            {
                /*
                 * If the peer device address is a random resolvable type address,
```

Freqchip Confidential

```
                 * the whitelist will only take effect when the peer device is added to RAL.
                */
                gap_add_device_to_resolve_list(&event->param.bond.peer_id_addr,
                                               event->param.bond.peer_irk, 1);
                gap_add_device_to_white_list(&event->param.bond.peer_id_addr, 1);
            }
            else
            {
                /*
                 * If the peer device address is a static(public / Private Static) type
address,
                 * just add it to the whitelist.
                 */
                gap_add_device_to_white_list(&event->param.bond.peer_id_addr, 1);
            }
        }
        break;
}
}
```

说明：

- 在 GAP 事件回调函数内，通过判断事件类型为 GAP_EVT_SMP_BOND_SUCCESS 来获取绑定成功事件处理的入口。
- 在绑定成功事件中 if((peer_device.addr_type == 0x01) && ((peer_device.addr.addr[5] & 0xC0) != 0xC0))用于判断对方设备地址是否是随机地址。如果是 resolved address 类型，执行 gap_add_device_to_resolve_list()和 gap_add_device_to_white_list()将设备添加到 RAL 和白名单；反之仅添加到白名单。

3. 广播设备使用白名单，广播可以通过设置广播参数中的过滤政策来使能白名单过滤功能。

```
gap_adv_param_t adv_param = {
        .own_addr_type = GAP_ADDR_TYPE_STATIC,
        .adv_mode = GAP_ADV_MODE_UNDIRECT,
        .disc_mode = GAP_ADV_DISC_MODE_GEN_DISC,
        .adv_chnl_map = GAP_ADV_CHAN_ALL,
        .filt_policy = GAP_ADV_FILTER_SCAN_WLST_CON_WLST,
        .phy_mode = GAP_PHY_TYPE_LE_1M,
        .adv_intv_min = 40,
        .adv_intv_max = 40,
    };
gap_adv_set_param(adv, &adv_param);
```

说明：

- 具体的广播发起流程参见 1.1.2 章节。

- 示例代码中 filt_policy 设置了广播的过滤选项为扫描和连接只响应白名单内的设备。设置之后，只有步骤 2 白名单内的设备能扫描和连接本广播。

1.3.3.2 地址配置说明

当开启广播、扫描、建立连接时，controller 会使用用户设置的本机地址发送空口数据包。本机地址在初始化时进行设置一次即可。广播、扫描、主动连接时均采用初始化时设置的本机地址做为设备地址。

1. BLE mac 地址类型，本机地址一共有 3 种类型，参见 gap_api.h 中的定义。

```
enum gap_own_addr_type {
    GAP_ADDR_TYPE_STATIC,
     //!< Public or Private Static Address according to device address configuration
    GAP_ADDR_TYPE_GEN_RSLV,
    //!< Generated resolvable private random address
    GAP_ADDR_TYPE_GEN_NON_RSLV,
    //!< Generated non-resolvable private random address
};
```

说明：

- GAP_ADDR_TYPE_STATIC 表示根据设备地址配置的公共或专用静态地址。
- GAP_ADDR_TYPE_GEN_RSLV 表示可解析私有随机地址，可以在一定程度上保护设备的隐私，无法长期被其他设备追踪。
- GAP_ADDR_TYPE_GEN_NON_RSLV 表示不可解析的专用随机地址，无法被解析为长期有效的地址，因此无法被其他设备追踪。

1.4 GAF

本节讲述 Ble Auido 相关 API 在 5090 SDK 上的使用方法，以下描述均基于一对一连接的情况下，即一个 Client 连接另一个 Server，并在此基础上进行 Ble Audio 相关 profile 的通信。可参考 SDK 里的 ble_audio demo。

1.4.1 BAP--Unicast Audio

BAP 中的 Unicast Audio 类由俩个服务组成，分别为 PACS(Published Audio Capabilities Service) 和 ASCS(Audio Stream Control Service)， PACS 服务用于向客户端暴露服务端支持的能力，ASCS 服务通过 ASE(Audio Stream Endpoint)进行音频流的控制，服务端可存在多个 PAC 和多个 ASE。

Unicast Client 只能由 GAP 中的 Central 角色担任。

Freqchip Confidential

Characteristic Name	Requirement	Mandatory Properties	Optional Properties	Security Permissions
Sink ASE	C.1	Read, Notify	None	Encryption required
Source ASE	C.1	Read, Notify	None	Encryption required
ASE Control Point	M	Write, WriteWithoutResponse, Notify	None	Encryption required

图 1-20 ASCS Characteristics

Characteristic Name	Requirement	Mandatory Properties	Optional Properties	Security Permissions
Sink PAC	C.1	Read	Notify	Encryption required
Sink Audio Locations	C.2	Read	Notify, Write	Encryption required
Source PAC	C.1	Read	Notify	Encryption required
Source Audio Locations	C.3	Read	Notify, Write	Encryption required
Available Audio Contexts	M	Read, Notify	None	Encryption required
Supported Audio Contexts	M	Read	Notify	Encryption required

图 1-21 PACS Characteristics

1.4.1.1 初始化

Client:

Client 端初始化代码如下:

```
uint32_t role_bf = GAF_BAP_ROLE_SUPP_UC_CAPA_CLI_BIT
                 | GAF_BAP_ROLE_SUPP_UC_ASCS_CLI_BIT;
  gaf_configure_param_t cfg_param;
  cfg_param.role_bf = role_bf;
  cfg_param.uc_ascs_cli_cfg.nb_ases_cfg = 1;

  gaf_cfg(gaf_callback, &cfg_param);
```

说明:

- Role_bf 表示本机将初始化的角色，上述代码里将本机初始为 ASCS、PACS 服务的 Client 端。
- Nb_ases_cfg 表示 ASCS 服务操作流程中，Client 端将要操作的 ASE 数目，上述代码里初始为 1。
- Gaf_callback 为 GAF 相关事件上报的回调函数。
- GAF 初始完后，以下事件将产生：

- GAF_BAP_CONFIGURED

Server：

Server 端初始化代码如下：

```
uint32_t role_bf = GAF_BAP_ROLE_SUPP_UC_CAPA_SRV_BIT
                   |GAF_BAP_ROLE_SUPP_UC_ASCS_SRV_BIT;

gaf_configure_param_t cfg_param;
cfg_param.role_bf = role_bf;
cfg_param.capa_srv_cfg.nb_pacs_sink = 0;
cfg_param.capa_srv_cfg.nb_pacs_src = 1;
cfg_param.capa_srv_cfg.location_bf_sink = 0;
cfg_param.capa_srv_cfg.location_bf_src = AUDIO_LOCA_BACK_RIGHT_BIT;
cfg_param.capa_srv_cfg.supp_context_bf_sink = 0;
cfg_param.capa_srv_cfg.supp_context_bf_src = CONTEXT_UNSPECIFIED_BIT;

cfg_param.uc_ascs_srv_cfg.nb_ase_chars = 1;
cfg_param.uc_ascs_srv_cfg.nb_ases_cfg = 1;
cfg_param.uc_ascs_srv_cfg.direction_bf = 0b1;

gaf_cfg(gaf_callback, &cfg_param);
```

说明:

- Role_bf 表示本机将初始化的角色，上述代码里将本机初始为 ASCS、PACS 服务的 Server 端。
- Cfg_param.capa_srv_cfg 表示 PACS 服务的相关配置，上述代码里配置了一个 Source PAC，Source Auido Location 配置为 BACK_RIGHT，Supported Audio Contexts 配置为 Unspecified。如果要配置多个 PAC 时，协议栈的 PAC 特征的排列顺序为 Sink 在前，Source 在后，如配置 2 个 Sink+1 个 Source，则特征表为 pac_list[3] = {Sink0, Sink1, Source0}。
- Cfg_param.uc_ascs_srv_cfg 表示 ASCS 服务的相关配置，上述代码里配置了一个 ASE，以及一个属于 ASE 特征的 CCC，且该 ASE 的为 Source 端。配置多个 ASE，如配置 1 个 Source ASE + 1 个 Sink ASE，且 Source 在前，则 nb_ase_chars = 2，nb_ases_cfg = 2，direction_bf = 0b01。Bit0 = 1，表示第一个 ASE 为 Source，bit1 = 0，表示第二个 ASE 为 Sink。
- GAF 初始完后，以下事件将产生：
  - GAF_EVT_BAP_CONFIGURED

Freechip

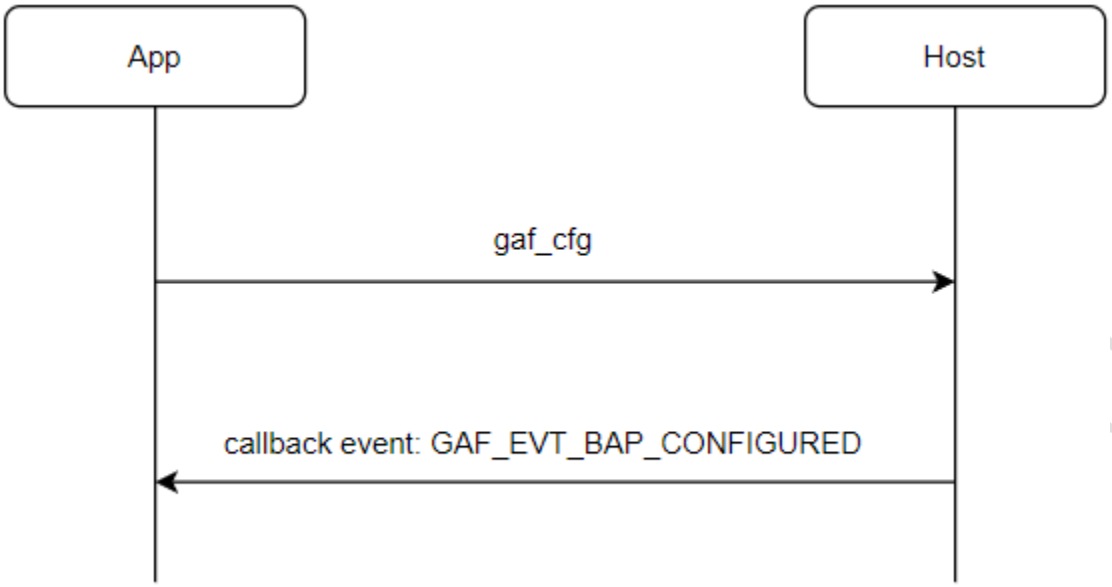


图 1-22 GAF 初始化流程图

BAP 层初始完后，需要设置 PAC 或者其他特征的特征值，下面以设置 PAC 的 Record 为例：

```
gaf_bap_uc_capa_srv_cmd_t cmd;
cmd.req_code = GAF_CMD_BAP_UC_CAPA_SRV_SET_RECORD;
cmd.p.set_record.record_id = 0;
cmd.p.set_record.pac_lid = 0;
cmd.p.set_record.codec_id.codec_id[4] = 0x00;
cmd.p.set_record.codec_id.codec_id[3] = 0x00;
cmd.p.set_record.codec_id.codec_id[2] = 0x00;
cmd.p.set_record.codec_id.codec_id[1] = 0x00;
cmd.p.set_record.codec_id.codec_id[0] = LC3;

gaf_bap_lc3_capa_t capa;
capa.sampling_freq_bf = SUPPROT_FREQ_48000_Hz_BIT;
capa.chan_cnt = CHANNEL_COUNT_1_BIT;
capa.frame_octet_min = 26;
capa.frame_octet_max = 120;
capa.context_bf = CONTEXT_UNSPECIFIED_BIT;
capa.frame_dur_bf = SUPPROT_FRAME_DURATION_10_ms_BIT;
capa.max_lc3_frame_sdu = 1;
capa.add_capa_len = 0;
capa.add_metadata_len = 0;

cmd.p.set_record.capa_len = sizeof(gaf_bap_lc3_capa_t);
cmd.p.set_record.capa = (uint8_t *)&capa;
```

```
gaf_bap_uc_capa_srv_cmd(&cmd);
```

说明：

- Pac_lid 表示配置的 PAC 的索引，如上述 Server 初始时配置 3 个 PAC 时，如果 pac_lid = 1，则配置的 PAC 为 Sink1。
- Record_id 表示设置的 record 的 ID，一个 PAC 可以有多个 record，但这些 record 的 record_id 不能相等。
- Codec_id 以及 capa 表示 codec 的相关参数，上述代码以配置 LC3 编码格式的 codec 为例。

至此，PACS 服务的初始化完成，用户也可基于上述代码，添加对 PACS 服务 Location 或者 Context 的设置。

1.4.1.2 发现服务

Client 端跟 Server 端建立 BLE 连接，且链路加密后，即可进行 BAP 的通信。下面讲述发现 PACS、ASCS 服务的流程，俩个服务发现流程可同时调用，也可单独调用。下面以先发现 PACS 服务，再发现 ASCS 服务为例。

Client:

```
gaf_bap_uc_capa_cli_cmd_t capa_cli_cmd;
capa_cli_cmd.cmd_code = GAF_CMD_BAP_UC_CAPA_CLI_DISCOVER;
capa_cli_cmd.p.disc.conidx = conidx;
gaf_bap_uc_capa_cli_cmd(&capa_cli_cmd);
```

上述为 Client 端发现 Server 端 PACS 服务的代码，发现过程完成后，下述事件将上报到 gaf_callback 中，事件对应描述请参考 gaf_api.h 中的注释:

1) GAF_EVT_BAP_UC_CAPA_CLI_BOND_DATA
2) GAF_EVT_BAP_UC_CAPA_CLI_RECORD
3) GAF_EVT_BAP_UC_CAPA_CLI_LOCATION
4) GAF_EVT_BAP_UC_CAPA_CLI_CONTEXT
5) GAF_EVT_BAP_UC_CAPA_CLI_CMD_CMP，
   cmd_code = GAF_CMD_BAP_UC_CAPA_CLI_DISCOVER

Freqchip Confidential

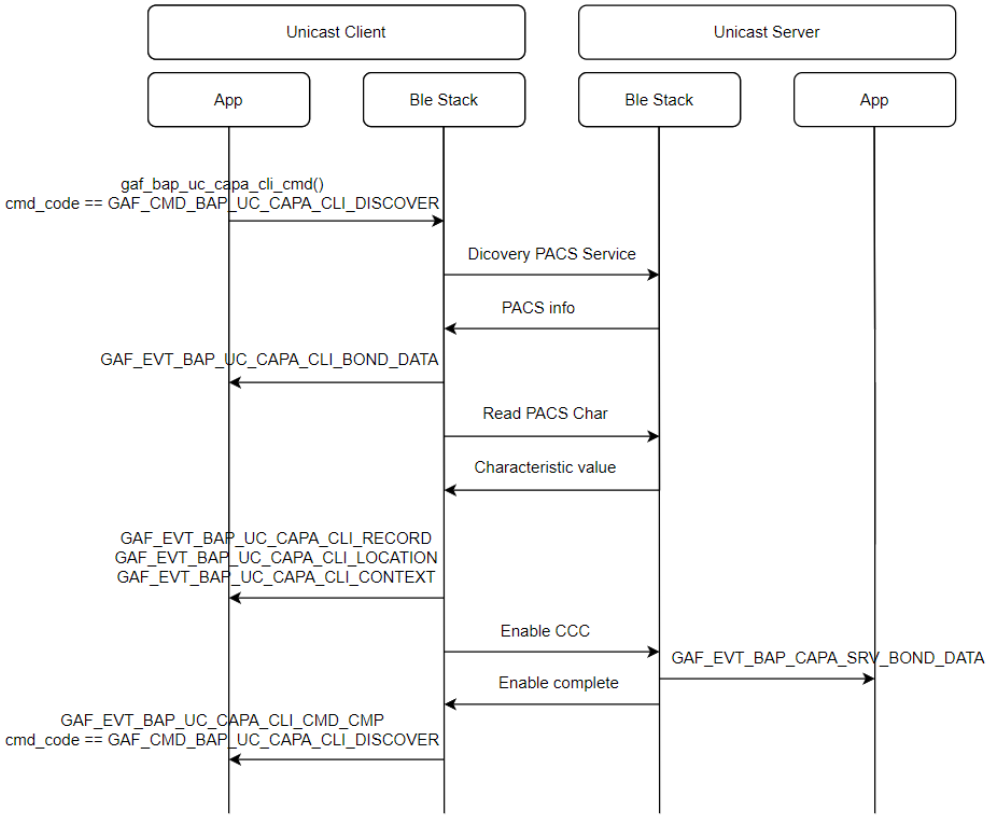


图 1-23 BAP/PACS 服务发现流程图

在 GAF_EVT_BAP_UC_CAPA_CLI_CMD_CMP 事件中，demo 中继续发现 ASCS 服务：

```
gaf_bap_uc_ascs_cli_cmd_t uc_ascs_cli_cmd;
uc_ascs_cli_cmd.cmd_code = GAF_CMD_BAP_UC_ASCS_CLI_DISCOVER;
uc_ascs_cli_cmd.p.disc.conidx = param->bap.capa_cli_cmd_cmp.p.discovery.conidx;
gaf_bap_uc_ascs_cli_cmd(&uc_ascs_cli_cmd);
```

上述为 Client 端发现 Server 端 ASCS 服务的代码，发现过程完成后，下述事件将上报到 gaf_callback 中，事件对应描述请参考 gaf_api.h 中的注释:

1) GAF_EVT_BAP_UC_ASCS_CLI_BOND_DATA
2) GAF_EVT_BAP_UC_ASCS_CLI_CFG
3) GAF_EVT_BAP_UC_ASCS_CLI_STATE_XXX(CODEC or QOS or EMPTY)

Server:

在 Client 端进行服务发现过程中，Server 端会产生以下事件:

1) GAF_EVT_BAP_CAPA_SRV_BOND_DATA
2) GAF_EVT_BAP_UC_ASCS_SRV_BOND_DATA

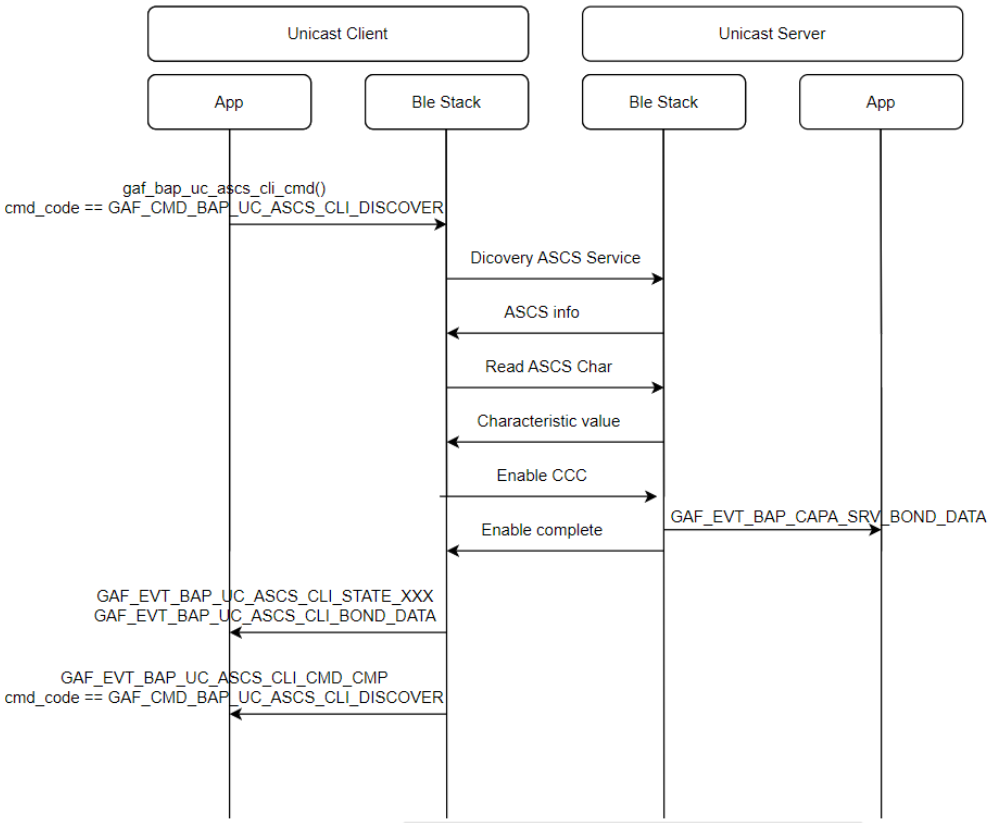


图 1-24 BAP/ASCS 服务发现流程图

1.4.1.3 ASE 操作流程

ASE 为 BAP 中控制音频流状态的特征，不同的操作会使 ASE 改变到不同状态，当 ASE 处于 Streaming 状态时，Client 与 Server 才可传输音频数据，ASE 的状态机如下图所示：

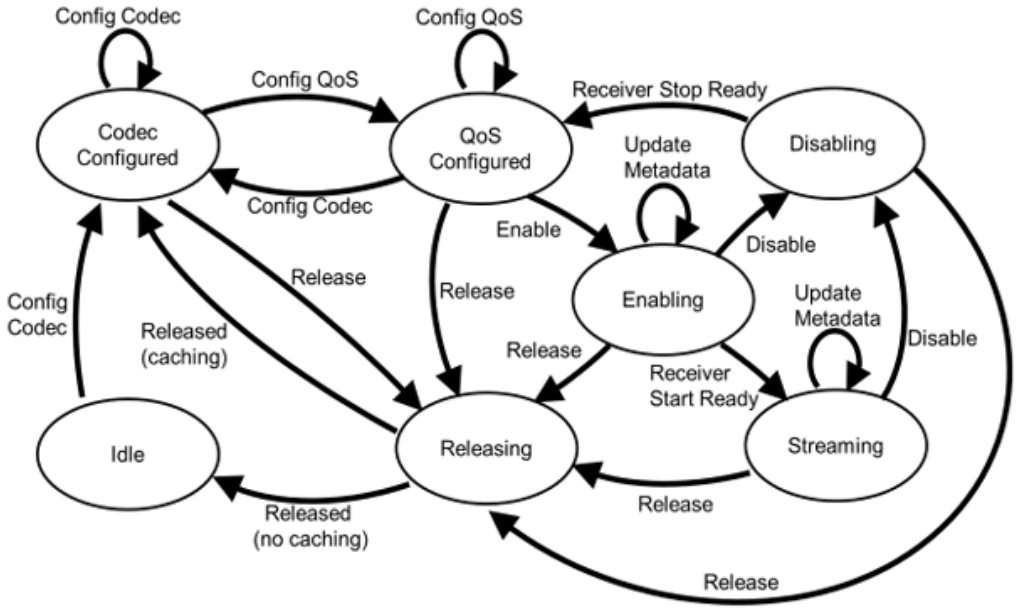


图 1-25 Source ASE 状态机

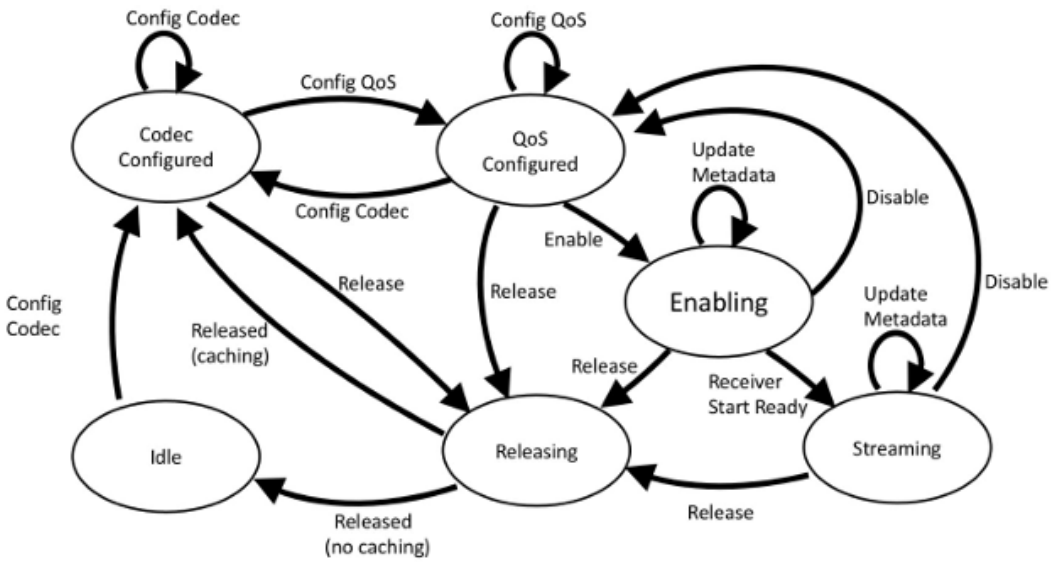


图 1-26 Sink ASE 状态机

1.4.1.4 Configure Codec

当 Client 发现服务流程进行完后，即可进行 Configure Codec 流程(Server 端 ASE 处于 IDLE、Cfg Codec、Cfg QOS)。

该流程中，Client 端和 Server 端将协商 Codec 的一些配置参数。流程完成后，Server 端对应 ASE 的 state 将变为 Codec Configured。

Client:

```
gaf_bap_uc_ascs_cli_cmd_t uc_cli_cmd;
uc_cli_cmd.cmd_code = GAF_CMD_BAP_UC_ASCS_CLI_CONFIGURE_CODEC;
uc_cli_cmd.p.cfg_codec.conidx = 0;
uc_cli_cmd.p.cfg_codec.ase_lid = 0;
uc_cli_cmd.p.cfg_codec.ase_instance_idx = 0;
uc_cli_cmd.p.cfg_codec.direction = GAF_HL_DIRECTION_SRC;
uc_cli_cmd.p.cfg_codec.tgt_latency = GAF_BAP_UC_TGT_LATENCY_BALENCED;
uc_cli_cmd.p.cfg_codec.tgt_phy = GAF_BAP_UC_TGT_PHY_2M;
uc_cli_cmd.p.cfg_codec.codec_id.codec_id[4] = 0;
uc_cli_cmd.p.cfg_codec.codec_id.codec_id[3] = 0;
uc_cli_cmd.p.cfg_codec.codec_id.codec_id[2] = 0;
uc_cli_cmd.p.cfg_codec.codec_id.codec_id[1] = 0;
uc_cli_cmd.p.cfg_codec.codec_id.codec_id[0] = LC3;

gaf_bap_lc3_cfg_t lc3_cfg;
```

```
lc3_cfg.params.sampling_freq = FREQ_48000_Hz;
lc3_cfg.params.frame_dur = FRAME_DURATION_10_ms;
lc3_cfg.params.frame_octet = 80;
lc3_cfg.params.location_bf = AUDIO_LOCA_BACK_RIGHT_BIT;
lc3_cfg.params.nb_lc3_frames = 1;
lc3_cfg.cfg_len = 0;
lc3_cfg.cfg = NULL;

uc_cli_cmd.p.cfg_codec.cfg_len = sizeof(gaf_bap_lc3_cfg_t) + lc3_cfg.cfg_len;
uc_cli_cmd.p.cfg_codec.cfg = (void *)&lc3_cfg;

gaf_bap_uc_ascs_cli_cmd(&uc_cli_cmd);
```

说明：

- 上述为 Client 端发起 CFG Codec 请求的代码。
- Ase_lid 为 Client 端本次操作的 ASE 分配的一个 Local ID，用于协议栈创建对应 ASE 的环境变量，index 的范围为 0 – nb_ases_cfg，nb_ases_cfg 为 Client 端初始化时设置的值。后续针对该 ASE 的操作都将以该 lid 为准。
- Ase_instance_index 为 Client 本次操作的 ASE 在服务端的 Index，该 Index 在事件 GAF_EVT_BAP_UC_ASCS_CLI_BOND_DATA 中上报。
- Direction 为操作的 ASE 的 direction，在事件 GAF_EVT_BAP_UC_ASCS_CLI_BOND_DATA 中上报。
- Tgt_phy 用于 Client 端与 Server 端协商后续建立 CIS 时使用的 PHY。
- 其余部分为 Codec 的参数配置，上述代码以 LC3 编码为例。
- Server 端同意配置请求后，Client 端将产生以下事件:
  - GAF_EVT_BAP_UC_ASCS_CLI_STATE_CODEC
  - GAF_EVT_BAP_UC_ASCS_CLI_CMD_CMP，cmd_code == GAF_CMD_BAP_UC_ASCS_CLI_CONFIGURE_CODEC

Server:

```
case GAF_EVT_BAP_UC_ASCS_SRV_CFG_CODEC_REQ:
{
  gaf_bap_uc_ascs_srv_cfm_t cfm;

  cfm.cfm_code = GAF_CFM_BAP_UC_ASCS_SRV_CONFIGURE_CODEC;
```

Freqchip Confidential

```
    cfm.p.cfg_codec.ase_lid = param->bap.uc_ascs_srv_cfg_codec_req.ase_lid;
    cfm.p.cfg_codec.rsp_code = BAP_UC_CP_RSP_CODE_SUCCESS;
    cfm.p.cfg_codec.reason = BAP_UC_CP_REASON_NO_ERROR;
    cfm.p.cfg_codec.qos_req.pref_pres_delay_min_us = 0;
    cfm.p.cfg_codec.qos_req.pref_pres_delay_max_us = 0;
    cfm.p.cfg_codec.qos_req.pres_delay_min_us = 0;
    cfm.p.cfg_codec.qos_req.pres_delay_max_us = 40000;
    cfm.p.cfg_codec.qos_req.trans_latency_max_ms = 20;
    cfm.p.cfg_codec.qos_req.framing = 0;
    cfm.p.cfg_codec.qos_req.phy_bf = param->bap.uc_ascs_srv_cfg_codec_req.tgt_phy;
    cfm.p.cfg_codec.qos_req.retx_nb = 3;

    memcpy((void*)&cfm.p.cfg_codec.codec_id,
    (void*)&param->bap.uc_ascs_srv_cfg_codec_req.codec_id, 5);

    gaf_bap_lc3_cfg_t lc3_cfg;
    lc3_cfg.params.sampling_freq = FREQ_48000_Hz;
    lc3_cfg.params.frame_dur = FRAME_DURATION_10_ms;
    lc3_cfg.params.frame_octet = MAX_SDU_SIZE;
    lc3_cfg.params.location_bf = AUDIO_LOCA_BACK_RIGHT_BIT;
    lc3_cfg.params.nb_lc3_frames = 1;
    lc3_cfg.cfg_len = 0;
    lc3_cfg.cfg = NULL;

    cfm.p.cfg_codec.cfg_len = sizeof(gaf_bap_lc3_cfg_t) + lc3_cfg.cfg_len;;
    cfm.p.cfg_codec.cfg = (void *)&lc3_cfg;

    gaf_bap_uc_ascs_srv_cfm(&cfm);
  }break;
```

说明：

- 上述代码为 Server 收到 Client 发起的 Configure Codec 请求后，产生 GAF_EVT_BAP_UC_ASCS_SRV_CFG_CODEC_REQ 事件，在本事件中，Server 端需要选择接受或者拒绝该请求。
- Rsp_code == BAP_UC_CP_RSP_CODE_SUCCESS 时，表示 Server 端接受该请求；Rsp_code 为其他值时，表示拒绝该请求。
- Qos_req 内的参数用于 Cfg QoS 阶段。

- 流程完成后，以下事件将产生：
  - GAF_EVT_BAP_UC_ASCS_SRV_ASE_STATE，state == GAF_EVT_BAP_UC_ASE_STATE_CODEC_CONFIGURED

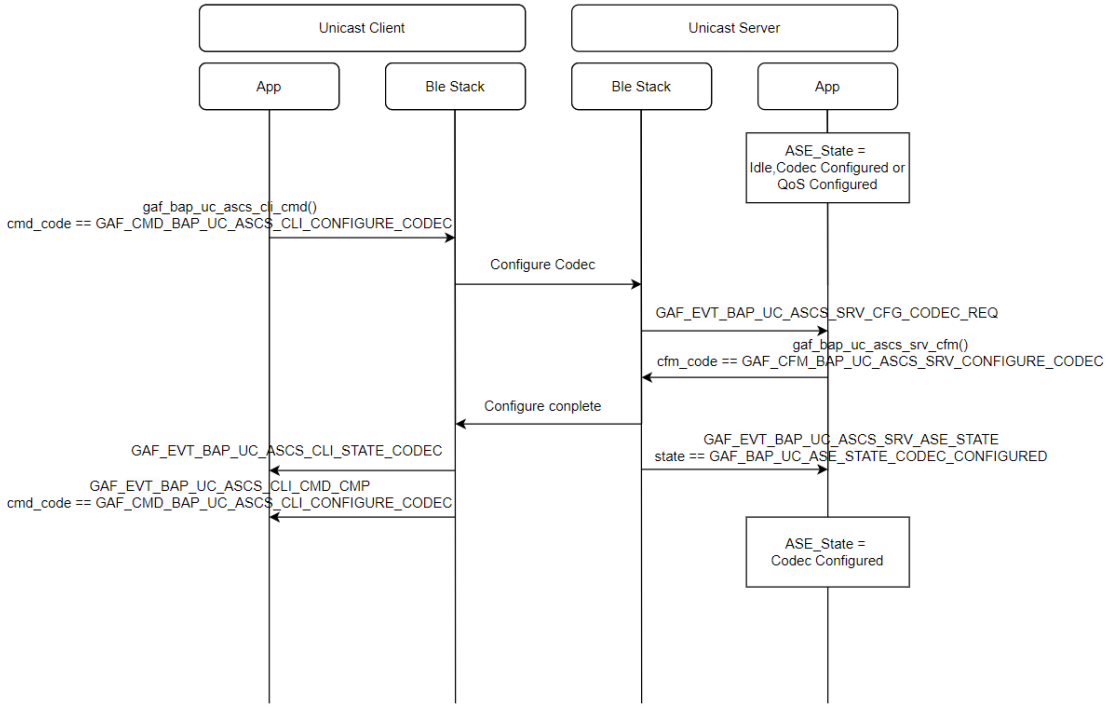


图 1-27 Configure Codec 流程图

1.4.1.5 Create CIG Group

在进行 QoS 之前，需要先创建一个 CIG(Connected Isochronous Group)，这是进行 Cfg Qos 之前必要的步骤，CIG 创建之后，能够一直使用，除非进行 remove group 的流程。

```
gaf_bap_uc_ascs_cli_cmd_t uc_ascs_cli_cmd;
uc_ascs_cli_cmd.cmd_code = GAF_CMD_BAP_UC_ASCS_CLI_CREATE_GROUP;
uc_ascs_cli_cmd.p.create_group.grp_lid = 0;
uc_ascs_cli_cmd.p.create_group.params.sdu_intv_m2s_us = 10000;
uc_ascs_cli_cmd.p.create_group.params.sdu_intv_s2m_us = 10000;
uc_ascs_cli_cmd.p.create_group.params.tlatency_m2s_ms = 20;
uc_ascs_cli_cmd.p.create_group.params.tlatency_s2m_ms = 20;
uc_ascs_cli_cmd.p.create_group.params.packing = 0;//Sequential
uc_ascs_cli_cmd.p.create_group.params.framing = 0;//unframed
uc_ascs_cli_cmd.p.create_group.params.sca = 0x01;

gaf_bap_uc_ascs_cli_cmd(&uc_ascs_cli_cmd);
```

说明：

- 上述为初始化 CIG 参数的代码，主要用于设置后续属于该 CIG 的 CIS(Connected Isochronous Stream)的链路参数。该操作可在 Cfg QoS 流程之前的任何阶段进行。
- 该操作完成后，将产生以下事件：
  - GAF_EVT_BAP_UC_ASCS_CLI_CMD_CMP

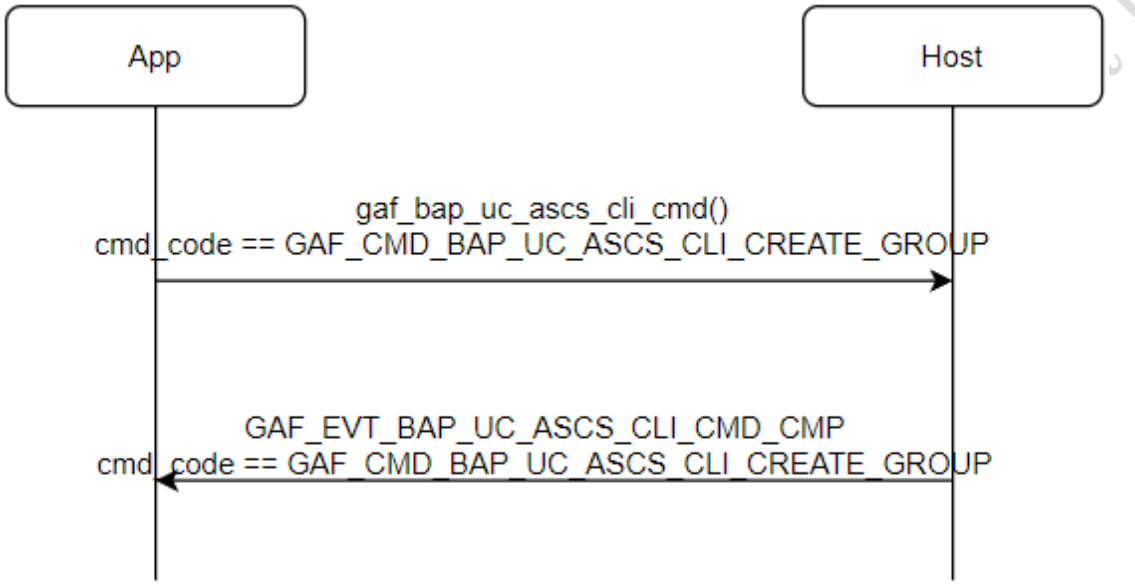


图 1-28 Create CIG 流程图

1.4.1.6 Configure QoS

CIG 参数初始完后，即可进行 Configure Qos 的流程，该流程用于给 ASE 分配 CIS 链路，以及该 CIS 链路所属的 CIG。

流程完成后，Server 端对应 ASE 的 state 将变为 Qos Configured。

Client:

```
gaf_bap_uc_ascs_cli_cmd_t uc_ascs_cli_cmd;
uc_ascs_cli_cmd.cmd_code = GAF_CMD_BAP_UC_ASCS_CLI_CONFIGURE_QOS;
uc_ascs_cli_cmd.p.cfg_qos.ase_lid = 0;
uc_ascs_cli_cmd.p.cfg_qos.grp_lid = 0;
uc_ascs_cli_cmd.p.cfg_qos.cis_id = 0;
uc_ascs_cli_cmd.p.cfg_qos.qos_cfg.max_sdu_size = 80;
uc_ascs_cli_cmd.p.cfg_qos.qos_cfg.phy = GAF_BAP_UC_TGT_PHY_2M;
uc_ascs_cli_cmd.p.cfg_qos.qos_cfg.pres_delay_us = 0;
uc_ascs_cli_cmd.p.cfg_qos.qos_cfg.retx_nb = 3;
```

```
gaf_bap_uc_ascs_cli_cmd(&uc_ascs_cli_cmd);
```

说明：

- Grp_lid 为上述 create group 操作时创建的 Local ID，表示该 CIS 链路所属的 CIG。
- Cis_id 为给该 ASE 分配的 CIS 链路的 ID。
- Max_sdu_size 表示一包音频数据帧数据的最大长度。
- Phy 表示该 CIS 链路通信时使用的通信速率为 2M。
- Pres_delay 用于多 CIS 链路通信时，音频数据的解码延时，用于优化多设备之间音频播放的同步性。
- Retx_nb 为 CIS 链路通信时，数据包的重传次数，即一个数据包的传输次数为(Retx_nb + 1)。
- 操作完成后，Client 端将产生以下事件，事件对应描述请参考 gaf_api.h 中的注释:
  - GAF_EVT_BAP_UC_ASCS_CLI_STATE_QOS
  - GAF_EVT_BAP_UC_ASCS_CLI_CMD_CMP， cmd_code == GAF_CMD_BAP_UC_ASCS_CLI_CONFIGURE_QOS

Server:

流程完成后，Server 端将产生以下事件:

- GAF_EVT_BAP_UC_ASCS_SRV_ASE_STATE，state == GAF_BAP_UC_ASE_STATE_QOS_CONFIGURED

注：同一设备的不同 direction 的 ASE 可以使用同一条 CIS，既一个 Source 和一个 Sink，但需要所有 ASE 在 Enable 操作之前都完成 Cfg Qos 操作。

注：同一 CIG 的多个 ASE，若都要进行 Enable 操作，则在 Enable 操作之前，该 CIG 的 ASE 都应完成 Cfg QoS 操作，否则当一个 ASE 已进入 Streaming 状态时，其余链路无法进行 Cfg QoS 的流程。

Freqchip Confidential

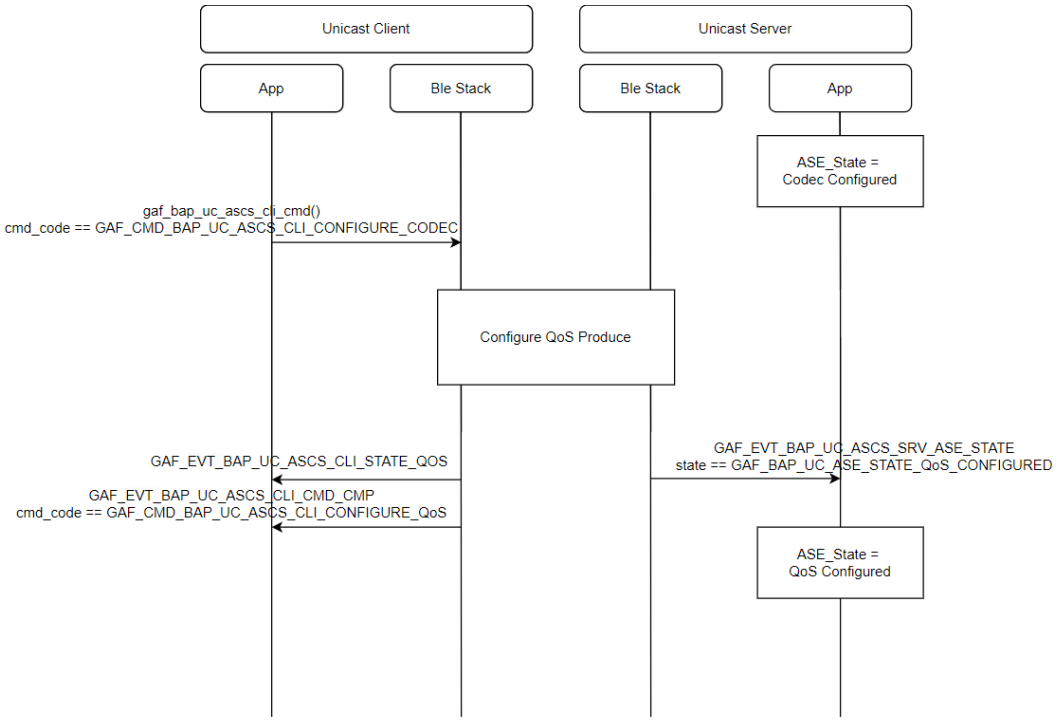


图 1-29 Configure QoS 流程图

1.4.1.7 Enable ASE

Configure Qos 操作完成后，即可进行开启音频流的操作。

Client:

```
gaf_bap_uc_ascs_cli_cmd_t uc_cli_cmd;
uc_cli_cmd.cmd_code = GAF_CMD_BAP_UC_ASCS_CLI_ASE_ENABLE;
uc_cli_cmd.p.ase_enable.ase_lid = 0;
uc_cli_cmd.p.ase_enable.metadata.context_bf = CONTEXT_UNSPECIFIED_BIT;
uc_cli_cmd.p.ase_enable.metadata.metadata_len = 0;
uc_cli_cmd.p.ase_enable.metadata.metadata = NULL;
gaf_bap_uc_ascs_cli_cmd(&uc_cli_cmd);
```

说明：

- 上述为 Client 端开启与 Server 端音频流的代码，操作完成后，Client 将产生以下事件，事件对应描述请参考 gaf_api.h 中的注释:
  - GAF_EVT_BAP_UC_ASCS_CLI_CIS_STATE，cis_state == GAF_BAP_UC_CIS_STATE_CON
  - GAF_EVT_BAP_UC_ASCS_CLI_CMD_CMP， cmd_code == GAF_CMD_BAP_UC_ASCS_CLI_ASE_ENABLE

Server:

```
case GAF_EVT_BAP_UC_ASCS_SRV_ASE_ENABLE_REQ:
{
  gaf_bap_uc_ascs_srv_cfm_t cfm;
  cfm.cfm_code = GAF_CFM_BAP_UC_ASCS_SRV_ASE_ENABLE;
  cfm.p.ase_enable.rsp_code = BAP_UC_CP_RSP_CODE_SUCCESS;
  cfm.p.ase_enable.reason = BAP_UC_CP_REASON_NO_ERROR;
  cfm.p.ase_enable.metadata.context_bf = CONTEXT_UNSPECIFIED_BIT;
  cfm.p.ase_enable.metadata.metadata_len = 0x00;
  cfm.p.ase_enable.metadata.metadata = NULL;

  gaf_bap_uc_ascs_srv_cfm(&cfm);
}break;
```

说明：

- 上述代码为 Server 收到 Client 发起的 Enable ASE 请求后，产生 GAF_EVT_BAP_UC_ASCS_SRV_ASE_ENABLE_REQ 事件，在本事件中，Server 端需要选择接受或者拒绝该请求。
- Rsp_code == BAP_UC_CP_RSP_CODE_SUCCESS 时，表示 Server 端接受该请求；Rsp_code 为其他值时，表示拒绝该请求。
- 操作完成后，Server 端将产生以下事件，事件对应描述请参考 gaf_api.h 中的注释:
  - GAF_EVT_BAP_UC_ASCS_SRV_CIS_STATE，cis_state == GAF_BAP_UC_CIS_STATE_CON
  - GAF_EVT_BAP_UC_ASCS_SRV_ASE_STATE，state == GAF_BAP_UC_ASE_STATE_STREAMING

当 Enable 流程完成，即 ASE 状态变为 Streaming 后，即可由 Source 发起音频数据的传输。当 Client 端为 Source，Server 端为 Sink 时，使用 gaf_bap_uc_cli_send_iso_data()发送数据，Sink 端收到数据后，将产生 GAF_EVT_BAP_UC_ASCS_SRV_ISO_DATA 事件；当 Server 端为 Source，Client 端为 Sink 时，使用 gaf_bap_uc_srv_send_iso_data()发送数据，Sink 端收到数据后，将产生 GAF_EVT_BAP_UC_ASCS_CLI_ISO_DATA 事件。

Freqchip Confidential

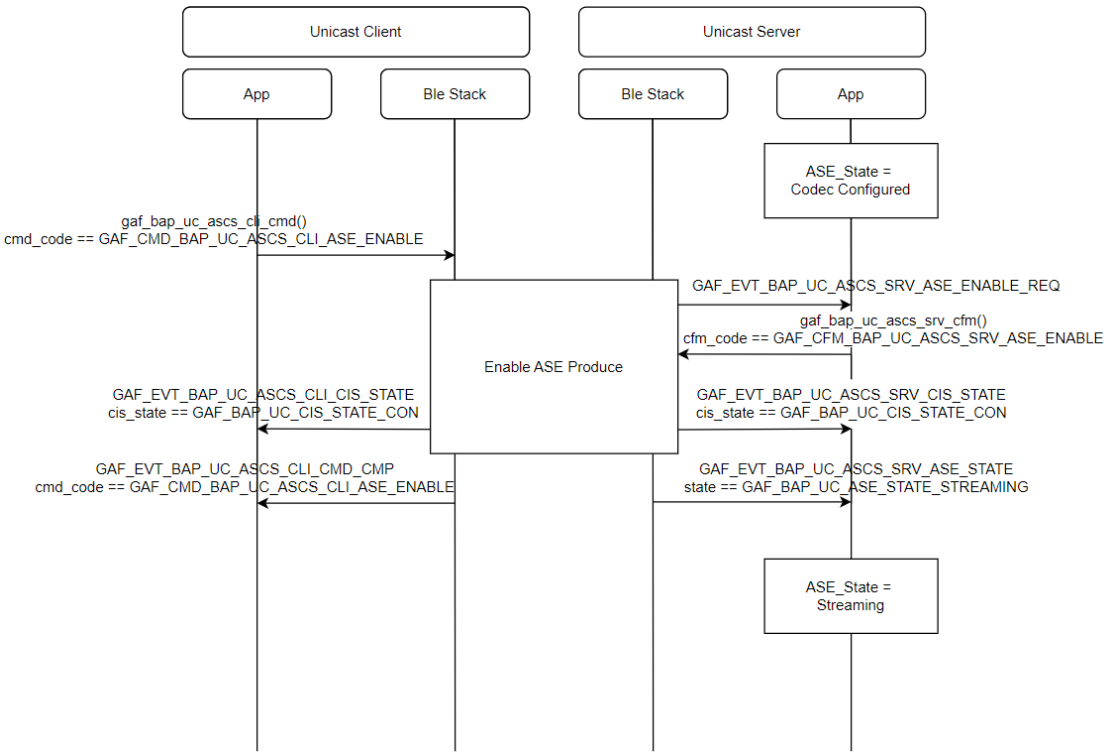


图 1-30 Enable ASE 流程图

1.4.1.8 Update Metadata

当 ASE 处于 Enabling 或者 Streaming 状态时，可以通过 Update Metadata 操作更新 Metadata。

1.4.1.9 Disable ASE

当 ASE 处于 Streaming 状态时，可以通过 Disable ASE 来关闭音频流。

该流程 Client 端与 Server 端都可以发起。下面描述 Client 端发起关闭音频流操作的流程。

Client:

```
gaf_bap_uc_ascs_cli_cmd_t uc_cli_cmd;
uc_cli_cmd.cmd_code = GAF_CMD_BAP_UC_ASCS_CLI_ASE_DISABLE;
uc_cli_cmd.p.ase_disable.ase_lid = 0;


gaf_bap_uc_ascs_cli_cmd(&uc_cli_cmd);
```

说明：

- 上述为 Client 端发起关闭音频流操作的代码，音频流关闭后，ASE 状态变为 Qos Configured 状态。
- 流程完成后，Client 端将产生以下事件：
  - GAF_EVT_BAP_UC_ASCS_CLI_CIS_STATE，cis_state == GAF_BAP_UC_CIS_STATE_LOST

- GAF_EVT_BAP_UC_ASCS_CLI_STATE_QOS
- GAF_EVT_BAP_UC_ASCS_CLI_CMD_CMP，cmd_code == GAF_BAP_UC_ASCS_CLI_CMD_ASE_DISABLE

Server:

流程完成后，Server 端将产生以下事件：

- GAF_EVT_BAP_UC_ASCS_SRV_ASE_STATE，state == GAF_BAP_UC_ASE_STATE_DISABLING
- GAF_EVT_BAP_UC_ASCS_SRV_CIS_STATE，cis_state == GAF_BAP_UC_CIS_STATE_LOST
- GAF_EVT_BAP_UC_ASCS_SRV_ASE_STATE，state == GAF_BAP_UC_ASE_STATE_QOS_CONFIGURED

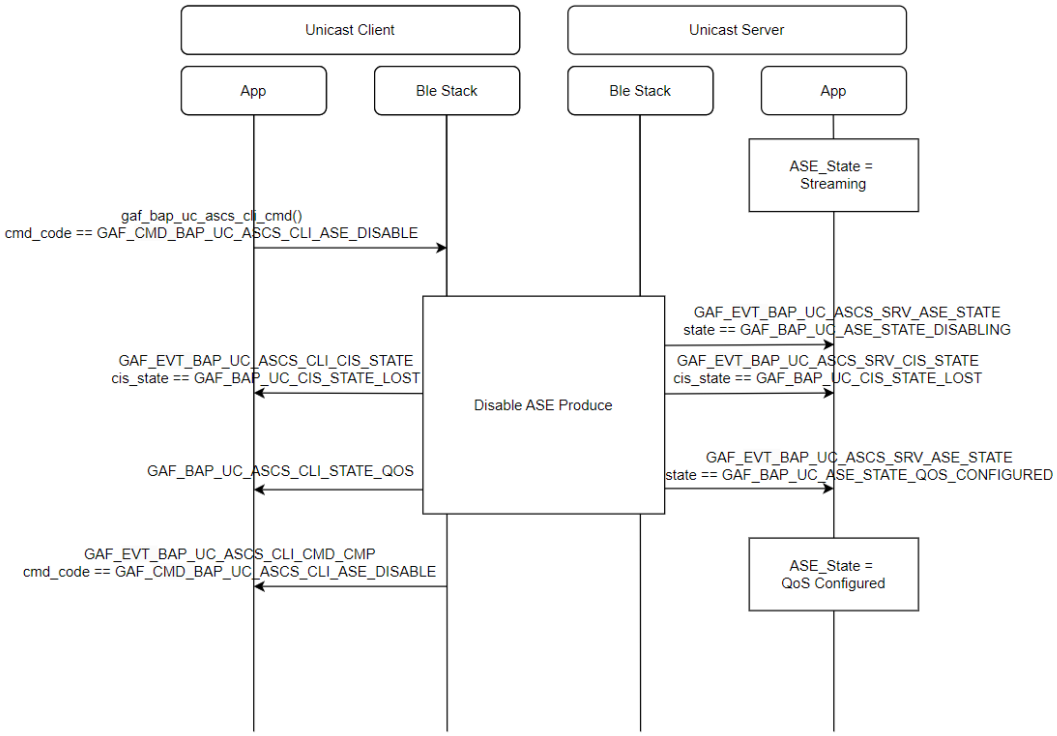


图 1-31 Disable ASE 流程图

1.4.1.10 Release ASE

当 ASE 处于 IDLE、Releasing 以外的其他任何状态时，都可通过 Release 流程让 ASE 变为 IDLE 或者 Codec Configured 状态。

该流程 Client 端与 Server 端均可发起。下面描述 Client 端发起 Release 操作的流程。

Client:

```
gaf_bap_uc_ascs_cli_cmd_t uc_cli_cmd;
uc_cli_cmd.cmd_code = GAF_CMD_BAP_UC_ASCS_CLI_ASE_RELEASE;
uc_cli_cmd.p.ase_disable.ase_lid = 0;
```

```
gaf_bap_uc_ascs_cli_cmd(&uc_cli_cmd);
```

说明：

- 上述为 Client 端发起 Release 操作的代码，操作完成后，ASE 的状态将变为 Server 端指定的状态 (IDLE or Codec Configured)。
- 流程完成后，Client 将产生以下事件：
  - GAF_EVT_BAP_UC_ASCS_CLI_STATE_EMPTY or GAF_BAP_UC_ASCS_CLI_STATE_CODEC
  - GAF_EVT_BAP_UC_ASCS_CLI_CMD_CMP， cmd_code == GAF_BAP_UC_ASCS_CLI_CMD_ASE_RELEASE
- 若 Release 前，ASE 处于 Streaming 态，Client 将产生以下事件:
  - GAF_EVT_BAP_UC_ASCS_CLI_CIS_STATE，cis_state == GAF_BAP_UC_CIS_STATE_LOST

Server:

```
case GAF_EVT_BAP_UC_ASCS_SRV_ASE_RELEASE_REQ:
{
  gaf_bap_uc_ascs_srv_cfm_t cfm;
  cfm.cfm_code = GAF_CFM_BAP_UC_ASCS_SRV_ASE_RELEASE;
  cfm.p.ase_release.rsp_code = BAP_UC_CP_RSP_CODE_SUCCESS;
  cfm.p.ase_release.reason = BAP_UC_CP_REASON_NO_ERROR;
  cfm.p.ase_release.idle = 1;
  gaf_bap_uc_ascs_srv_cfm(&cfm);
}break;
```

说明：

- 上述代码为 Server 收到 Client 发起的 Release ASE 请求后，产生 GAF_EVT_BAP_UC_ASCS_SRV_ASE_RELEASE_REQ 事件，在本事件中，Server 端需要选择接受或者拒绝该请求。
- Rsp_code == BAP_UC_CP_RSP_CODE_SUCCESS 时，表示 Server 端接受该请求；Rsp_code 为其他值时，表示拒绝该请求。
- Idle 表示流程完成后，对应 ASE 的指定状态，为 1 代表 ASE 变为 IDLE 态，为 0 代表 ASE 变为 Codec Configured 态。
- 流程完成后，Server 将产生以下事件：
  - GAF_EVT_BAP_UC_ASCS_SRV_ASE_STATE，state == GAF_BAP_UC_ASE_STATE_RELEASING

Freqchip Confidential

- GAF_EVT_BAP_UC_ASCS_SRV_ASE_STATE，state == GAF_BAP_UC_ASE_STATE_IDLE or CODEC_COFIGURED

- 若 Release 前，ASE 处于 Streaming 态，Server 将产生以下事件:
  - GAF_EVT_BAP_UC_ASCS_SRV_CIS_STATE，cis_state == GAF_BAP_UC_CIS_STATE_LOST

注:当有多个 ASE 使用同一 CIS，且均处于 streaming 状态时，不能对其中任意一个 ASE 进行 Release 操作，需要先进行 Disable 流程后，再进行后续流程。

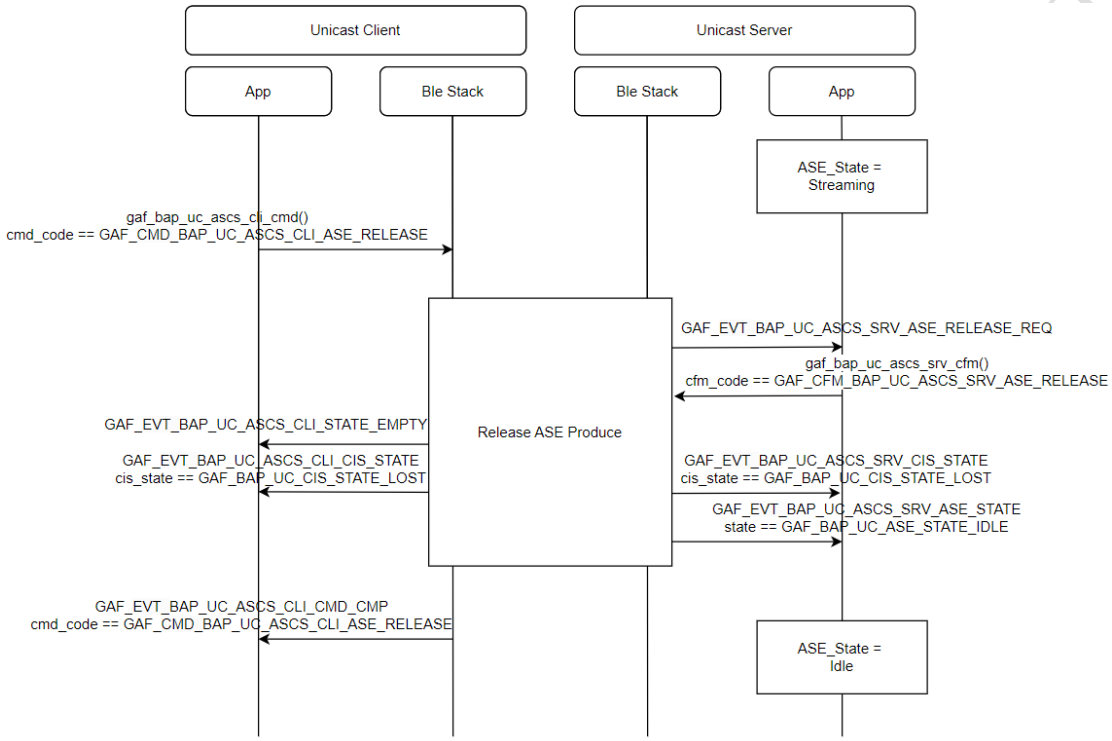


图 1-32 Release ASE 流程图

1.4.1.11 回连时重载服务信息

当 Client 与 Server 端进行服务发现流程时，Client 端可以在 GAF_EVT_BAP_UC_CAPA_CLI_BOND_DATA 和 GAF_EVT_BAP_UC_ASCS_CLI_BOND_DATA 事件中存储 Server 端的 PACS 服务和 ASCS 服务的内容，Server 端可以在 GAF_EVT_BAP_UC_CAPA_SRV_BOND_DATA 和 GAF_EVT_BAP_UC_ASCS_SRV_BOND_DATA 事件中存储 Client 端对该服务 CCC 的配置值。当 Client 与 Server 回连时，可以使用重载服务操作来跳过发现服务的流程。

Client:

```
void app_gaf_cli_restore_bd(uint8_t conidx)
{
    app_ble_audio_env_t *p_audio_env = &audio_env[conidx];
```

```
    p_audio_env->restore_cnt = 0;

    gaf_bap_uc_capa_cli_cmd_t capa_cli_cmd;
    capa_cli_cmd.cmd_code = GAF_CMD_BAP_UC_CAPA_CLI_RESTORE_BOND_DATA;
    capa_cli_cmd.p.restore_bd.conidx = conidx;
    memcpy(&capa_cli_cmd.p.restore_bd.pacs_info, &p_audio_env->pacs_info,
    sizeof(gaf_bap_uc_pacs_info_t));
    gaf_bap_uc_capa_cli_cmd(&capa_cli_cmd);

    gaf_bap_uc_ascs_cli_cmd_t ascs_cli_cmd;
    ascs_cli_cmd.cmd_code = GAF_CMD_BAP_UC_ASCS_CLI_RESTORE_BOND_DATA;
    ascs_cli_cmd.p.restore_bd.conidx = conidx;
    memcpy(&ascs_cli_cmd.p.restore_bd.ascs_info, &p_audio_env->ascs_info,
sizeof(gaf_bap_uc_ascs_info_t));
    gaf_bap_uc_ascs_cli_cmd(&ascs_cli_cmd);
}
```

说明：

- 上述为 Client 端重载 Server 服务的代码，重载成功后，Client 将产生以下事件：
  - GAF_EVT_BAP_UC_CAPA_CLI_REQ_CMP， req_code == GAF_BAP_UC_CAPA_CLI_REQ_RESTORE_BOND_DATA
  - GAF_EVT_BAP_UC_ASCS_CLI_REQ_CMP，req_code == GAF_BAP_UC_ASCS_CLI_REQ_RESTORE_BOND_DATA

Server:

```
void app_gaf_srv_restore_bd(uint8_t conidx)
{
    gaf_bap_uc_capa_srv_cmd_t capa_srv_cmd;
    capa_srv_cmd.cmd_code = GAF_CMD_BAP_UC_CAPA_SRV_RESTORE_BOND_DATA;
    capa_srv_cmd.p.restore_bd.conidx = conidx;
    capa_srv_cmd.p.restore_bd.cli_cfg_bf = audio_env.pacs_cli_cfg_bf;
    capa_srv_cmd.p.restore_bd.pac_cli_cfg_bf = audio_env.pac_cli_cfg_bf;
    capa_srv_cmd.p.restore_bd.evt_bf = capa_srv_req.p.restore_bd.cli_cfg_bf;
    capa_srv_cmd.p.restore_bd.pac_evt_bf = capa_srv_req.p.restore_bd.pac_cli_cfg_bf;
    capa_srv_cmd.p.restore_bd.context_bf_sink = 0;
    capa_srv_cmd.p.restore_bd.context_bf_src = CONTEXT_UNSPECIFIED_BIT;

    gaf_bap_uc_capa_srv_cmd(&capa_srv_cmd);
```

```
    gaf_bap_uc_ascs_srv_cmd_t ascs_srv_cmd;
    ascs_srv_cmd.cmd_code = GAF_CMD_BAP_UC_ASCS_SRV_RESTORE_BOND_DATA;
    ascs_srv_cmd.p.restore_bd.conidx = conidx;
    ascs_srv_cmd.p.restore_bd.cp_cli_cfg = audio_env.ase_cp_cli_cfg;
    ascs_srv_cmd.p.restore_bd.ase_cli_cfg_bf = audio_env.ase_cli_cfg_bf;

    gaf_bap_uc_ascs_srv_cmd(&ascs_srv_cmd);
}
```

说明：

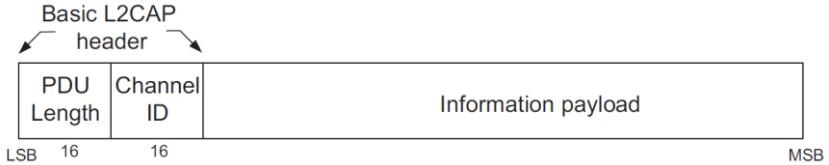
- 上述为 Server 端重载本地服务 CCC 配置的代码。
- evt_bf 和 pac_evt_bf 为控制本地在重载完服务配置后，是否发送 notification 给 Client 端，告知 Client 端本地 PACS 服务的相关数据。
- 重载成功后，Server 端将产生以下事件:
  - GAF_EVT_BAP_UC_CAPA_SRV_REQ_CMP，req_code == GAF_BAP_UC_CAPA_SRV_REQ_RESTORE_BOND_DATA

GAF_EVT_BAP_UC_ASCS_SRV_REQ_CMP，req_code == GAF_BAP_UC_ASCS_SRV_REQ_RESTORE_BOND_DATA

1.5 L2CAP COC

1.5.1 背景介绍

L2CAP 是所有 Host 数据传输的载体，可以进行组包、拆包，流量控制等功能，它的基本数据格式是：



Basic information frame (B-frame)

其中 CID（Channel ID）用于指示这个数据包属于哪一种上层服务，比如 ATT、SMP 等。CID 由 SIG 定义，分成两种类型，固定和动态：

CID	分配功能
0x0000	不能使用
0x0001~0x0003	预留
0x0004	ATT
0x0005	BLE 指令信道

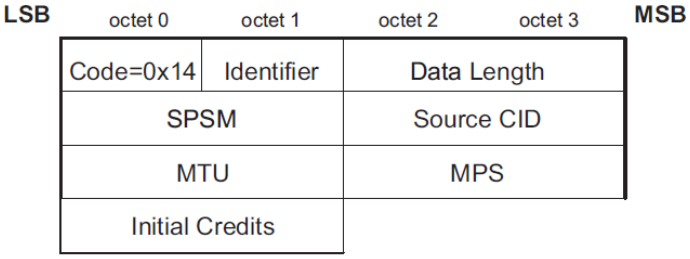
Freqchip Confidential

0x0006	SMP
0x0007~0x001f	预留
0x0020~0x003e	SIG 定义
0x003f	预留
0x0040~0x007f	动态分配，用于 L2CAP LE credit based create Connection 和 L2CAP credit based create Connection

L2CAP LE credit based create Connection 是一种支持流控的连接，为了支持这种功能，SIG 定义了面向连接的信道 COC（Connection Oriented Channel）来创建这种连接。

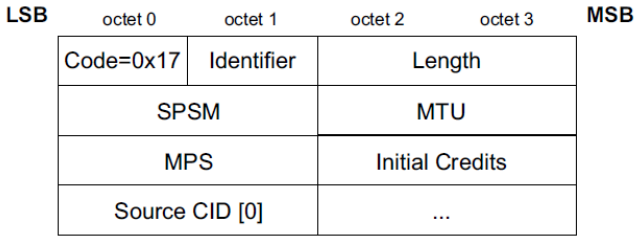
COC 允许用户在指定的链路上创建专用的信道，服务端和客户端在交换任何数据之前都需要先创建一个 COC 信道。COC 的最大优势是可以通过配置 MTU 和 MPS 使得应用层可以发送长包数据，以便提高系统吞吐率，典型的应用为 Internet Protocol Support Profile（IPSP）以及 Object Transfer Profile（OTP），另外新版本协议中添加的 Enhanced ATT 也是基于该链路。

SIG 定义了一组指令（通过 BLE 指令信道传输）来创建和销毁 COC，对于创建 L2CAP LE credit based create Connection，基本的指令格式为：



通过该指令用户可以创建一条 COC 信道用于 L2CAP LE credit based create Connection 的数据传输。

SIG 同样定义了指令来创建 L2CAP credit based create Connection，这种连接目前主要用于 Ehanced ATT，用户可以简单做个了解，可以不做重点关注，指令格式是：



通过该指令用户创建了过掉 COC 信道用于 L2CAP credit based create Connection 的数据传输。

上面介绍的两条指令中设计几个关键的参数：

1. SPSM：

   创建 COC 信道需要双方都支持，并且双方都注册了对应的服务，SPSM 便是用来指证服务类型，SIG 对 SPSM 进行了定义：

SPSM	类型	用途
0x0001~0x007f	固定的，SIG 分配	用于固定的用途，IPSP、OTP 等应用
0x0080~0x00ff	动态	由应用指定或者 GATT 注册
其他	预留	

2. MTU：

   在这条连接上可以传输的最大的应用层包长；

3. MPS：

   在这条连接上可以接收的最大 COC 数据包长；

4. Source CID

   给该连接分配的信道编号，动态分配。

1.5.2 API 说明

本节介绍 COC 相关的一些 API，除此之外，用户可以参照 example 中的 ble_l2cap_coc 例程，实际测试 COC 的建立过程和数据的收发。

1.5.2.1 创建和销毁服务

1. 注册回调

在初始化的时候用于需要调用如下 API，用于注册回调函数，用来接收异步消息：

```
void l2cap_coc_init(l2cap_coc_evt_cb_t cb);
```

2. 创建服务

前面提到，如果需要建立这种信道，服务端需要在应用层声明对对应服务的支持，这样在接收到对端发起的连接请求时就能正常做出相应，否则信道无法成功创建。

```
void l2cap_coc_spsm_new(uint16_t spsm, uint8_t sec_lvl);
```

这个 API 用于对服务的声明，可以在初始化的时候调用：

spsm：前面提到这个表示服务类型，可以支持的范围是 0x0080~0x00ff；

sec_lvl：表示建立服务的时候，对当前链路安全状态的需求：

位	功能
BIT[7:3]	预留
BIT[2:1]	安全状态需求： 0- 无要求 1- 加密，未鉴权 2- 加密，鉴权 3- 安全连接
BIT[0]	密钥长度： 0- 无要求 1- 128-bits

3. 删除服务

Freqchip Confidential

```
void l2cap_coc_spsm_delete(uint16_t spsm);
```

该 API 用于删除指定服务。

1.5.2.2 信道控制

1. 建立连接

在 ACL 连接建立好之后，客户端可以调用如下 API 向服务端发起建立 COC 的请求，建立成功之后会在回调函数中接收到相应的消息：

```
void l2cap_coc_connect(uint8_t conidx, uint8_t nb_ch, uint16_t spsm, uint16_t rx_mtu);
```

conidx：当前连接的索引；

nb_ch：创建几条 COC 信道，当前配置为 1 即可；

spsm：连接对端的哪项服务，该项服务需要对端已经注册好，如果连接对端不存在的服务，则会创建失败；

rx_mtu：本机支持最大的接收包长。

2. 重新配置 MTU

在 COC 建立好之后，用户可以调整支持的 MTU，该 API 支持同时修改多条 COC 的 MTU：

```
void l2cap_coc_reconfig(uint8_t conidx, uint16_t rx_mtu_new, uint8_t nb_ch, uint8_t *chan_lids);
```

conidx：当前连接的索引；

rx_mtu_new：需要更新的本机支持最大的接收包长；

nb_ch：修改几条 COC 信道的参数；

chan_lids：这是一个数组，长度为 nb_ch，保存 COC 信道的索引，这个索引来自 COC 建立好之后产生的回调。

3. 断开连接

应用层可以选择主动断开已经建立好的 COC 信道：

```
void l2cap_coc_disconnect(uint8_t conidx, uint8_t chan_lid);
```

4. 响应连接请求

提供服务的一端，在收到对端的连接请求时，需要调用如下的函数做出响应：

```
void l2cap_coc_conn_cfm(uint16_t token, uint8_t conidx, uint8_t nb_chan, uint16_t local_rx_mtu);
```

token：与 L2CAP_COC_EVT_CONNECT_REQ 中的保持一致即可；

conidx：当前连接的索引；

nb_chan：L2CAP_COC_EVT_CONNECT_REQ 中也包含该字段，用户层决定接收多少条连接；

local_rx_mtu：本机的 MTU 值。

Freqchip Confidential

1.5.2.3 发送数据

应用层可以调用如下 API 向对端发送数据，数据的最大长度不超过对端的 RX_MTU（这个在 L2CAP_COC_EVT_CONNECTED 消息中可以获得）：

```
void l2cap_coc_send_data(uint8_t conidx, uint8_t chan_lid, uint16_t len, uint8_t *data);
```

Freqchip Confidential

联系信息

公司：上海富芮坤微电子有限公司

地址：中国(上海)自由贸易试验区碧波路 912 弄 8 号 501-A 室

电话：+86-21-5027-0080

网址: www.freqchip.com

销售邮箱: sales@freqchip.com

文档邮箱: docs@freqchip.com

本文档的所有部分，其著作产权归上海富芮坤微电子有限公司（简称富芮坤）所有，未经富芮坤授权许可，任何个人及组织不得复制、转载、仿制本文档的全部或部分。富芮坤保留在不另行通知的情况下随时对产品或本文档进行更改、修正、增强的权利。购买者应在订购前获得富芮坤产品的最新相关资料。

Freqchip Confidential